Deep Reinforcement Learning

Unit 2: Introduction to Q-Learning

Hugging Face Deep RL Course
Faithful MDX-to-LaTeX study edition

Original content: Hugging Face Deep RL Course.

Contents

1 Introduction to Q-Learning **4**

2 What is RL? A short recap **5**

3 Two types of value-based methods **6**
3.1 The state-value function . . . 9
3.2 The action-value function . . . 9

4 The Bellman Equation: simplify our value estimation **11**

5 Monte Carlo vs Temporal Difference Learning **14**
5.1 Monte Carlo: learning at the end of the episode . . . 14
5.2 Temporal Difference Learning: learning at each step . . . 18

6 Mid-way Recap **21**

7 Mid-way Quiz **22**
7.1 Answer Key and Explanations . . . 23

8 Introducing Q-Learning **25**
8.1 What is Q-Learning? . . . 25
8.2 The Q-Learning algorithm . . . 29
8.3 Off-policy vs On-policy . . . 32

9 A Q-Learning example **33**
9.1 Step 1: Initialize the Q-table . . . 35
9.2 Step 2: Choose an action using the Epsilon Greedy Strategy . . . 35
9.3 Step 3: Perform action At, get Rt+1 and St+1 . . . 35
9.4 Step 4: Update Q(St, At) . . . 36
9.5 Step 2: Choose an action using the Epsilon Greedy Strategy . . . 37
9.6 Step 3: Perform action At, get Rt+1 and St+1 . . . 37
9.7 Step 4: Update Q(St, At) . . . 38

10 Q-Learning Recap **38**

11 Hands-on **40**

12 Unit 2: Q-Learning with FrozenLake-v1 and Taxi-v3 **41**
12.1 Objectives of this notebook . . . 42
12.2 This notebook is from the Deep Reinforcement Learning Course . . . 43
12.3 Prerequisites . . . 43
12.4 A small recap of Q-Learning . . . 43

13 Let's code our first Reinforcement Learning algorithm **45**
13.1 Install dependencies and create a virtual display . . . 45
13.2 Import the packages . . . 46

14 Part 1: Frozen Lake (non slippery version) **46**
14.1 ## Create and understand FrozenLake environment . . . 46
14.2 Create and Initialize the Q-table . . . 48
14.3 Define the greedy policy . . . 50
14.4 Define the epsilon-greedy policy . . . 50
14.5 Define the hyperparameters . . . 51
14.6 Create the training loop method . . . 52
14.7 Train the Q-Learning agent . . . 53
14.8 Let's see what our Q-Learning table looks like now . . . 54
14.9 The evaluation method . . . 54
14.10Evaluate our Q-Learning agent . . . 54
14.11Publish our trained model to the Hub . . . 55

15 Part 2: Taxi-v3 **59**
15.1 ## Create and understand Taxi-v3 . . . 59
15.2 Define the hyperparameters . . . 61
15.3 Train our Q-Learning agent . . . 62
15.4 Create a model dictionary and publish our trained model to the Hub . . . 63

16 Part 3: Load from Hub **63**
16.1 Some additional challenges . . . 65
16.2 Keep learning, stay awesome . . . 66

17 Second Quiz **66**
17.1 Answer Key and Explanations . . . 67

18 Glossary **69**

19 Conclusion **70**

20 Additional Readings **71**

20.1 Monte Carlo and TD Learning 71

20.2 Q-Learning 71

1 Introduction to Q-Learning



Figure 1: Unit 2 thumbnail [original figure]

In the first unit of this class, we learned about Reinforcement Learning (RL), the RL process, and the different methods to solve an RL problem. We also **trained our first agents and uploaded them to the Hugging Face Hub.**

In this unit, we're going to **dive deeper into one of the Reinforcement Learning methods: value-based methods** and study our first RL algorithm: **Q-Learning.**

We'll also **implement our first RL agent from scratch**, a Q-Learning agent, and will train it in two environments:

1. Frozen-Lake-v1 (non-slippery version): where our agent will need to **go from the starting state (S) to the goal state (G)** by walking only on frozen tiles (F) and avoiding holes (H).
2. An autonomous taxi: where our agent will need **to learn to navigate** a city to **transport its passengers from point A to point B.**

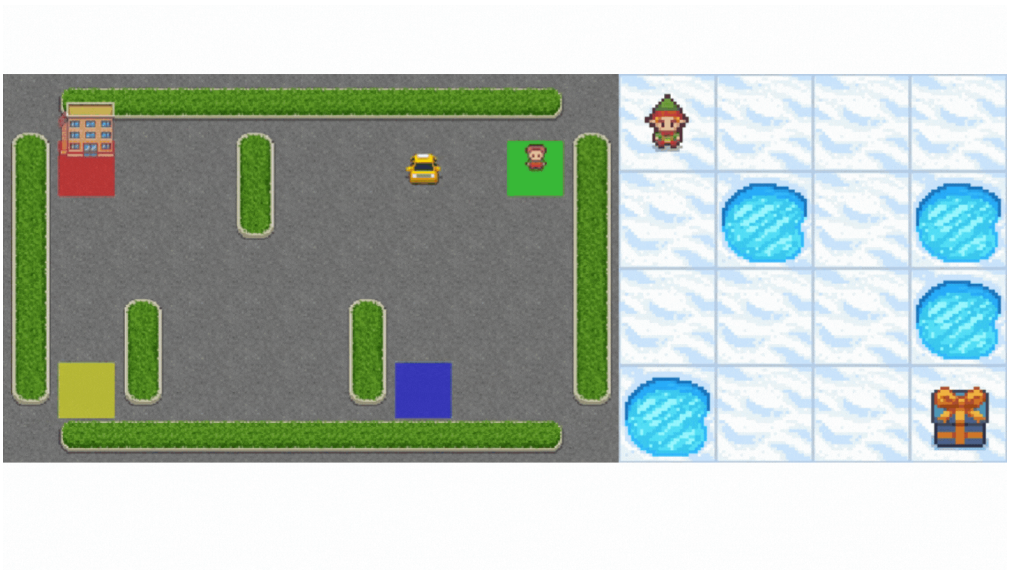


Figure 2: envs [original animation]

Concretely, we will:

- Learn about **value-based methods**.

- Learn about the **differences between Monte Carlo and Temporal Difference Learning**.
- Study and implement **our first RL algorithm**: Q-Learning.

This unit is **fundamental if you want to be able to work on Deep Q-Learning**: the first Deep RL algorithm that played Atari games and beat the human level on some of them (breakout, space invaders, etc).

So let's get started!

2 What is RL? A short recap

In RL, we build an agent that can **make smart decisions**. For instance, an agent that **learns to play a video game.** Or a trading agent that **learns to maximize its benefits** by deciding on **what stocks to buy and when to sell.**

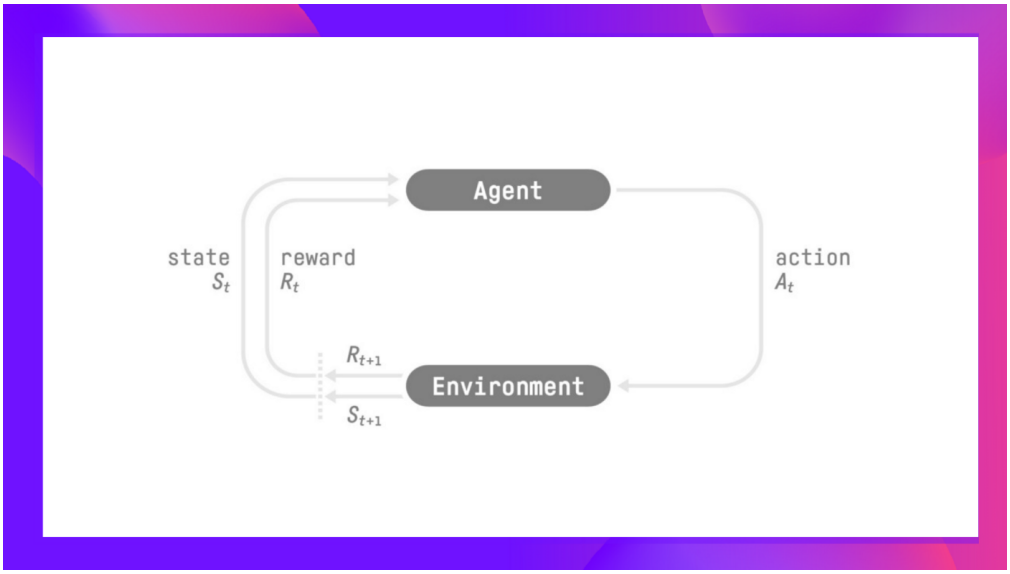


Figure 3: The agent-environment feedback loop [original figure]

To make intelligent decisions, our agent will learn from the environment by **interacting with it through trial and error** and receiving rewards (positive or negative) **as unique feedback.**

Its goal **is to maximize its expected cumulative reward** (because of the reward hypothesis).

The agent's decision-making process is called the policy π: given a state, a policy will output an action or a probability distribution over actions. That is, given an observation of the environment, a policy will provide an action (or multiple probabilities for each action) that the agent should take.



Figure 4: A policy maps observations to actions [original figure]

Our goal is to find an optimal policy π^* , aka., a policy that leads to the best expected cumulative reward.

And to find this optimal policy (hence solving the RL problem), there **are two main types of RL methods**:

- *Policy-based methods*: **Train the policy directly** to learn which action to take given a state.
- *Value-based methods*: **Train a value function** to learn **which state is more valuable** and use this value function **to take the action that leads to it.**

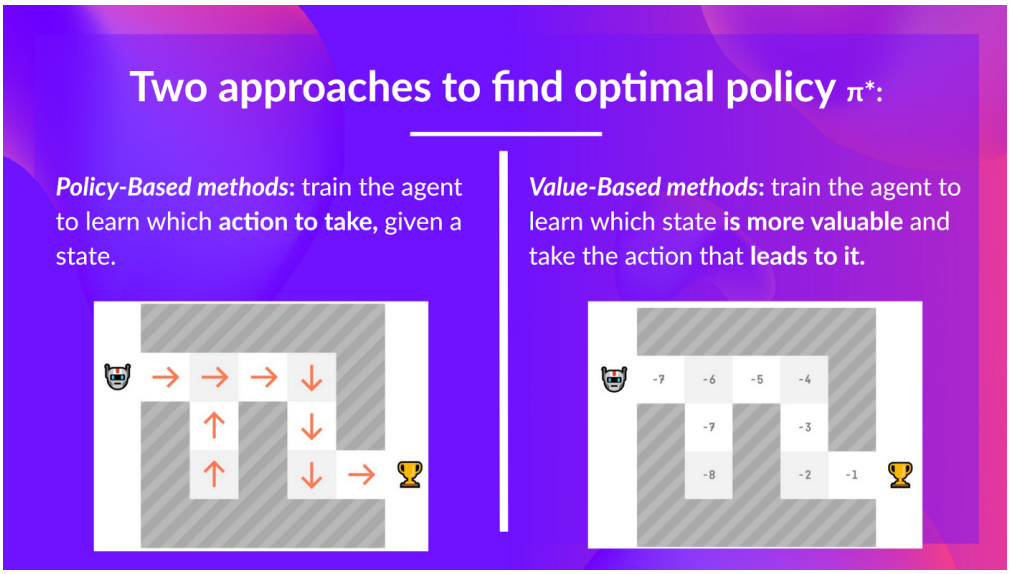


Figure 5: The two fundamental approaches: direct policy learning and value-function learning [original figure]

And in this unit, **we'll dive deeper into the value-based methods.**

3 Two types of value-based methods

In value-based methods, **we learn a value function** that **maps a state to the expected value of being at that state.**

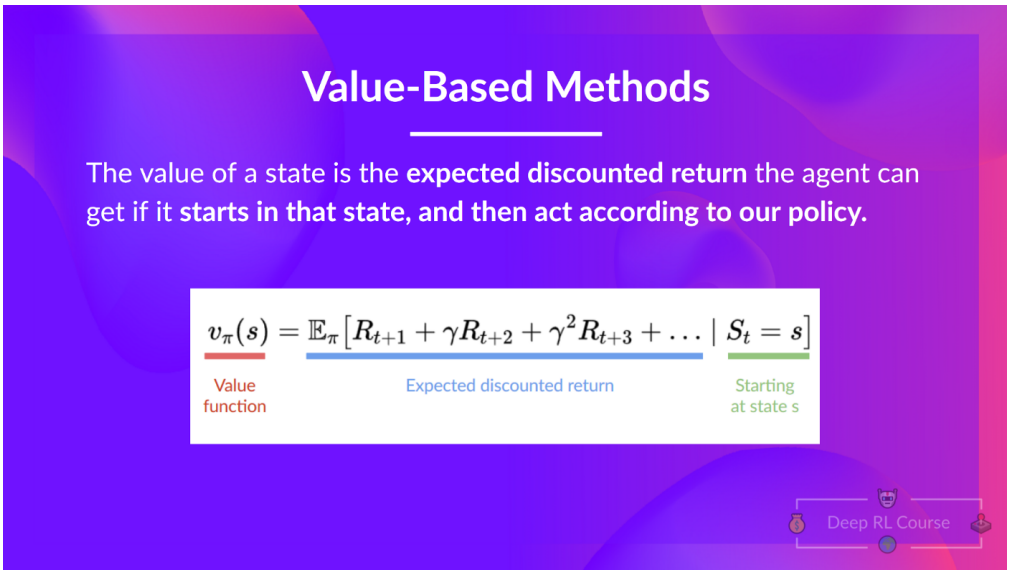


Figure 6: Overview of value-based reinforcement learning [original figure]

The value of a state is the **expected discounted return** the agent can get if it **starts at that state and then acts according to our policy.**

> **Course note.** But what does it mean to act according to our policy? After all, we don't have a policy in value-based methods since we train a value function and not a policy.

Remember that the goal of an **RL agent is to have an optimal policy π^*.**

To find the optimal policy, we learned about two different methods:

- *Policy-based methods:* **Directly train the policy** to select what action to take given a state (or a probability distribution over actions at that state). In this case, we **don't have a value function.**

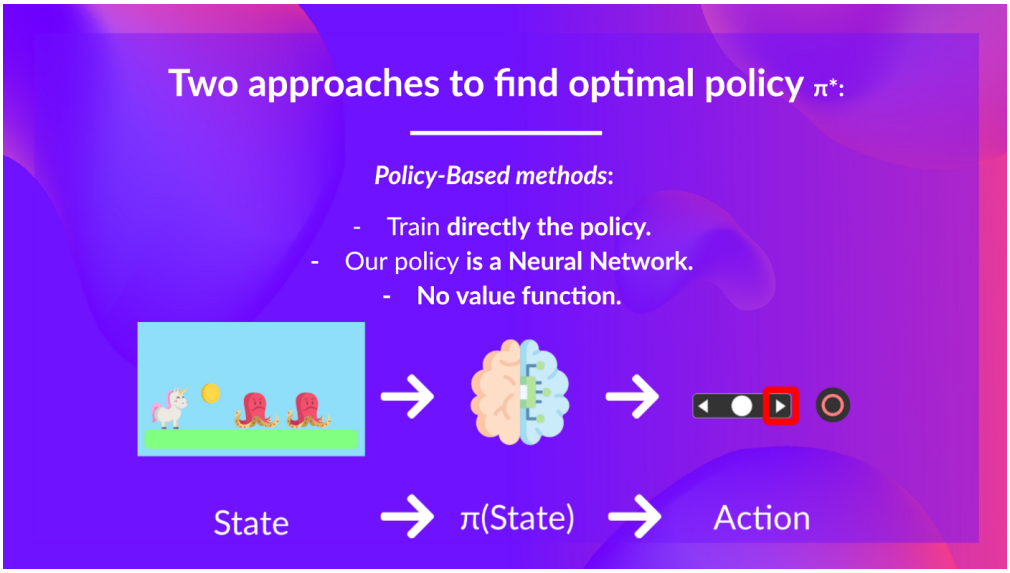


Figure 7: Policy-based learning: actions predicted directly by a policy [original figure]

The policy takes a state as input and outputs what action to take at that state (deterministic policy: a policy that output one action given a state, contrary to stochastic policy that output a probability distribution over actions).

And consequently, **we don't define by hand the behavior of our policy; it's the training that will define it.**

- *Value-based methods:* **Indirectly, by training a value function** that outputs the value of a state or a state-action pair. Given this value function, our policy **will take an action.**

Since the policy is not trained/learned, **we need to specify its behavior.** For instance, if we want a policy that, given the value function, will take actions that always lead to the biggest reward, **we'll create a Greedy Policy.**

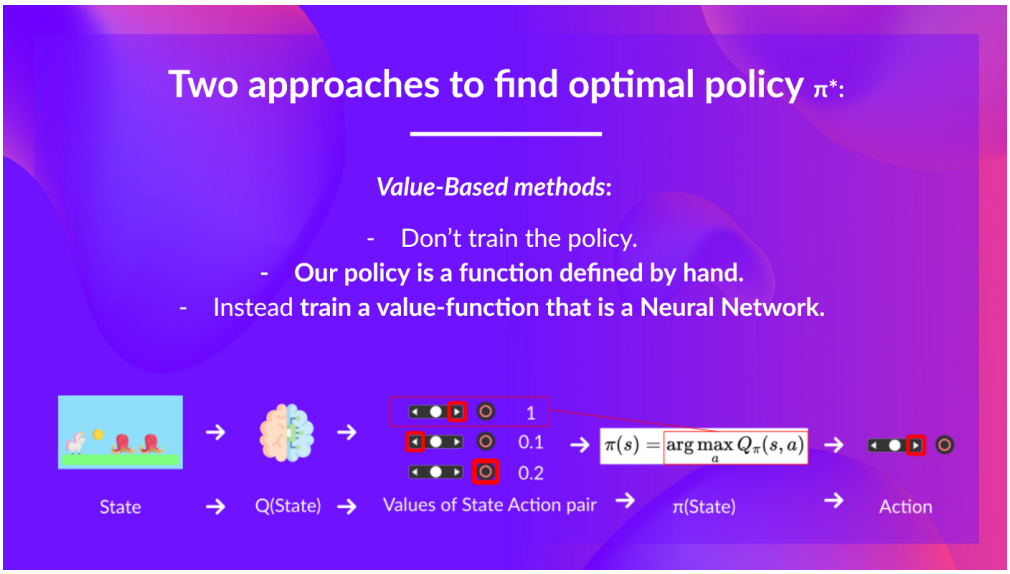


Figure 8: Given a state, our action-value function (that we train) outputs the value of each action at that state. Then, our pre-defined Greedy Policy selects the action that will yield the highest value given a state or a state action pair. [original figure]

Consequently, whatever method you use to solve your problem, **you will have a policy**. In the case of value-based methods, you don't train the policy: your policy **is just a simple pre-specified function** (for instance, the Greedy Policy) that uses the values given by the value-function to select its actions.

So the difference is:

- In policy-based training, **the optimal policy (denoted π^*) is found by training the policy directly.**
- In value-based training, **finding an optimal value function (denoted Q* or V*, we'll study the difference below) leads to having an optimal policy.**

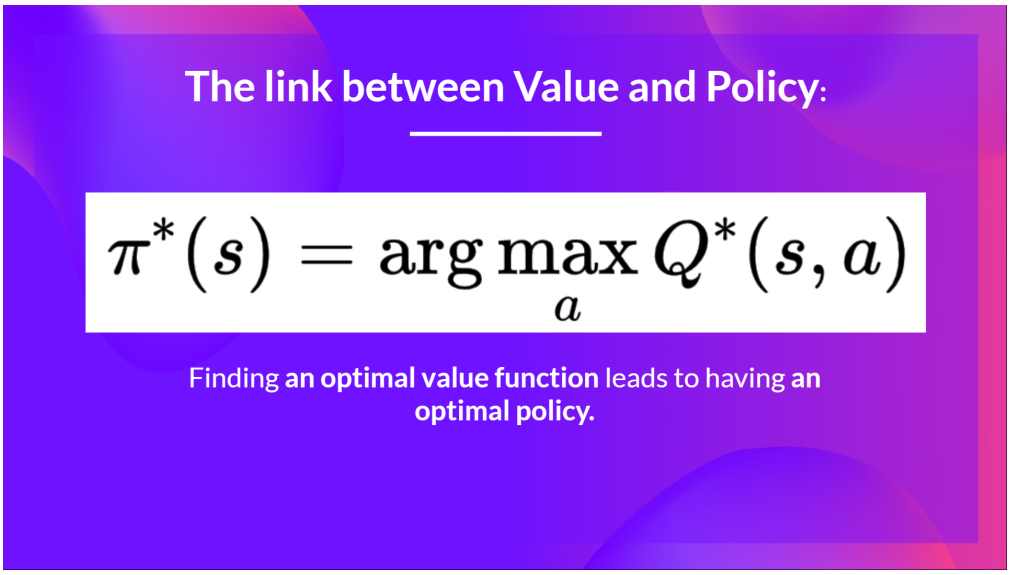


Figure 9: Recovering an optimal policy from the optimal action-value function [original figure]

In fact, most of the time, in value-based methods, you'll use **an Epsilon-Greedy Policy** that handles the exploration/exploitation trade-off; we'll talk about this when we talk about Q-Learning in the second part of this unit.

As we mentioned above, we have two types of value-based functions:

3.1 The state-value function

We write the state value function under a policy π like this:

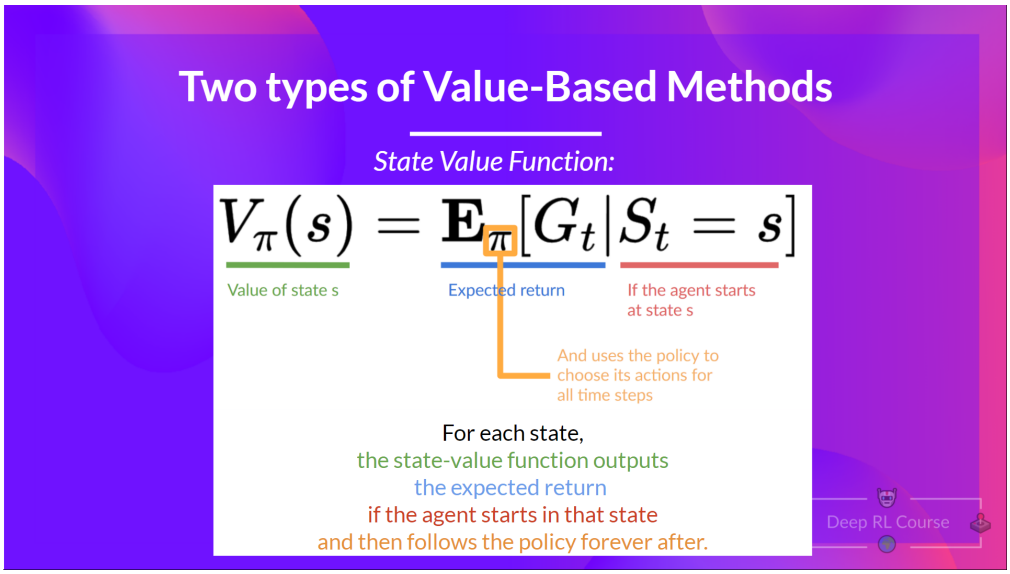


Figure 10: Mathematical definition of the state-value function [original figure]

For each state, the state-value function outputs the expected return if the agent **starts at that state** and then follows the policy forever afterward (for all future timesteps, if you prefer).

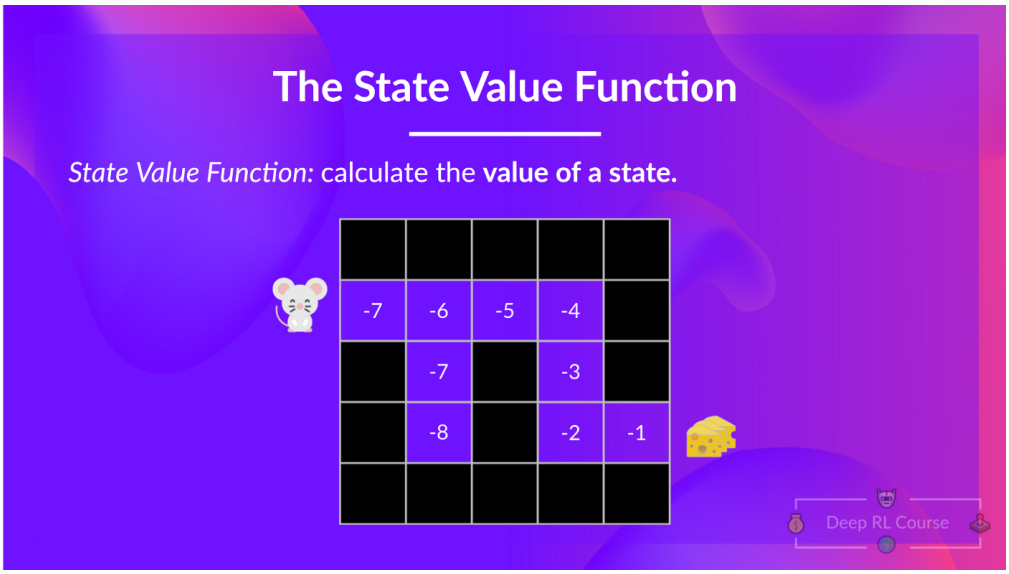


Figure 11: If we take the state with value -7: it's the expected return starting at that state and taking actions according to our policy (greedy policy), so right, right, right, down, down, right, right. [original figure]

3.2 The action-value function

In the action-value function, for each state and action pair, the action-value function **outputs the expected return** if the agent starts in that state, takes that action, and then follows the policy forever after.

The value of taking action a in state s under a policy π is:

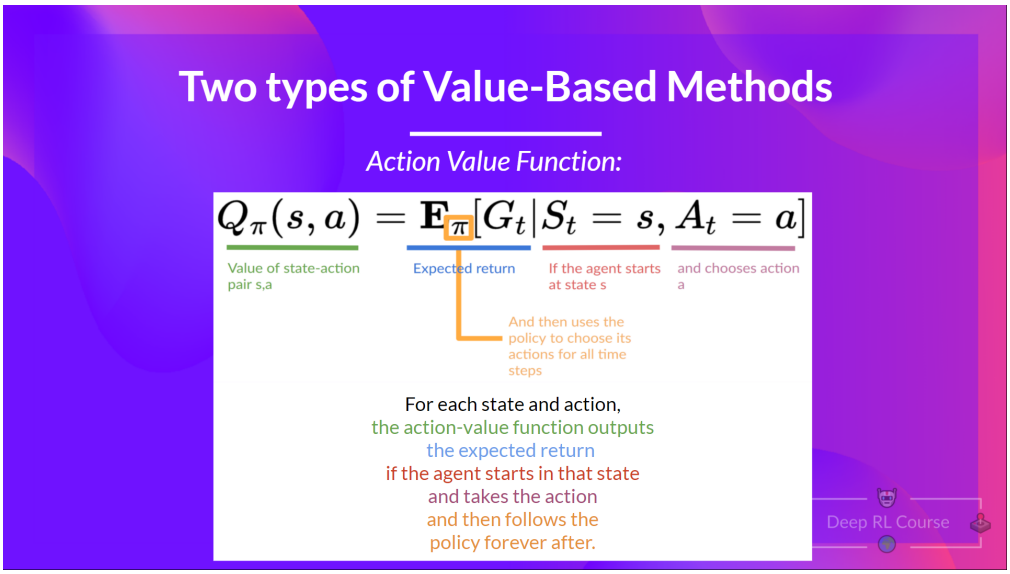


Figure 12: Mathematical definition of the action-value function [original figure]

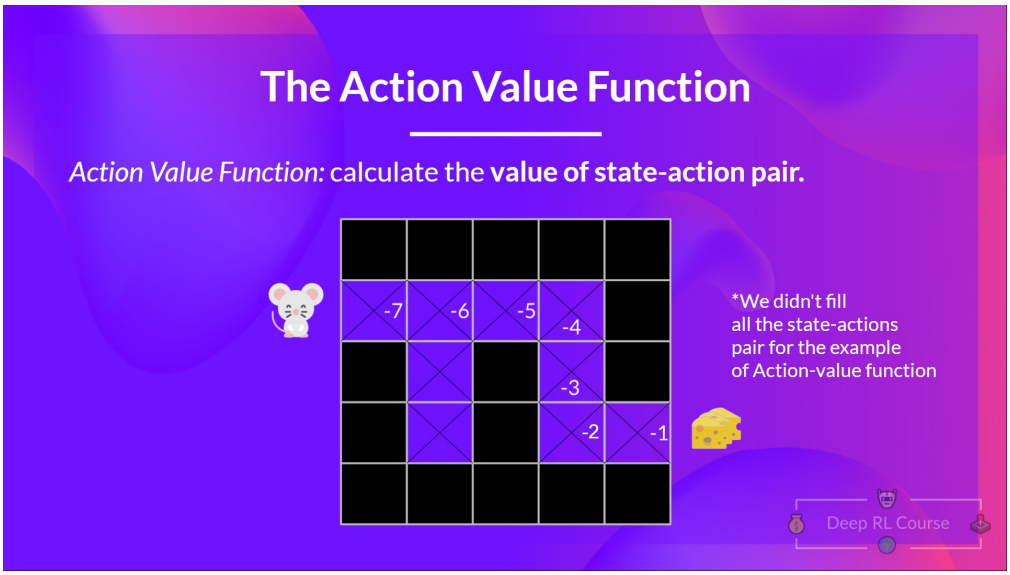


Figure 13: Action values for a chosen state and action [original figure]

We see that the difference is:

- For the state-value function, we calculate **the value of a state** S_t
- For the action-value function, we calculate **the value of the state-action pair** (S_t, A_t) **hence the value of taking that action at that state.**

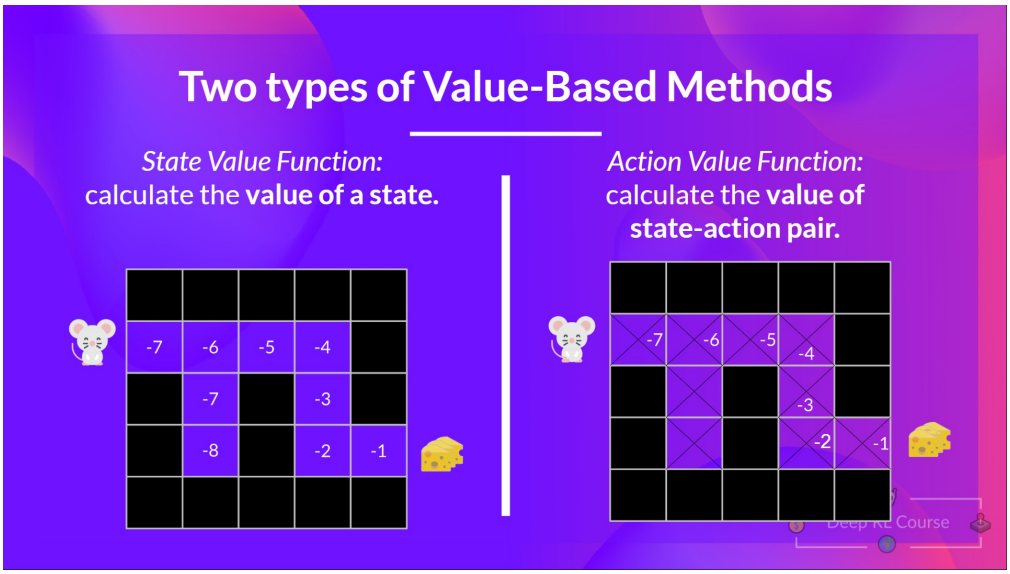


Figure 14: Note: We didn't fill all the state-action pairs for the example of Action-value function [original figure]

In either case, whichever value function we choose (state-value or action-value function), **the returned value is the expected return.**

However, the problem is that **to calculate EACH value of a state or a state-action pair, we need to sum all the rewards an agent can get if it starts at that state.**

This can be a computationally expensive process, and that's **where the Bellman equation comes in to help us.**

4 The Bellman Equation: simplify our value estimation

The Bellman equation **simplifies our state value or state-action value calculation.**

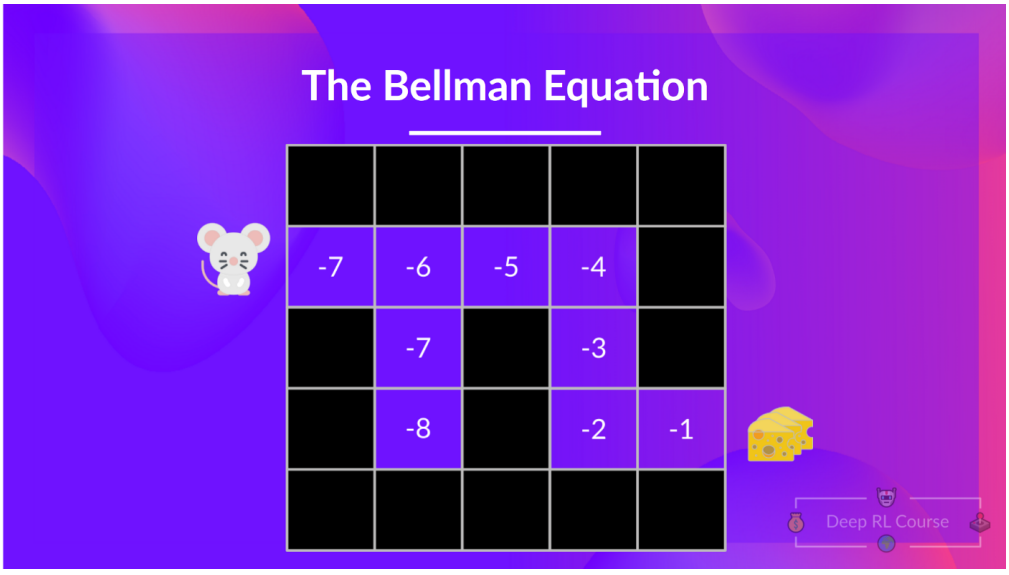


Figure 15: Bellman equation and recursive value estimation [original figure]

With what we have learned so far, we know that if we calculate $V(S_t)$ (the value of a state), we need to calculate the return starting at that state and then follow the policy forever after. **(The policy we defined in the following example is a Greedy Policy; for simplification, we don't discount the reward).**

So to calculate $V(S_t)$, we need to calculate the sum of the expected rewards. Hence:

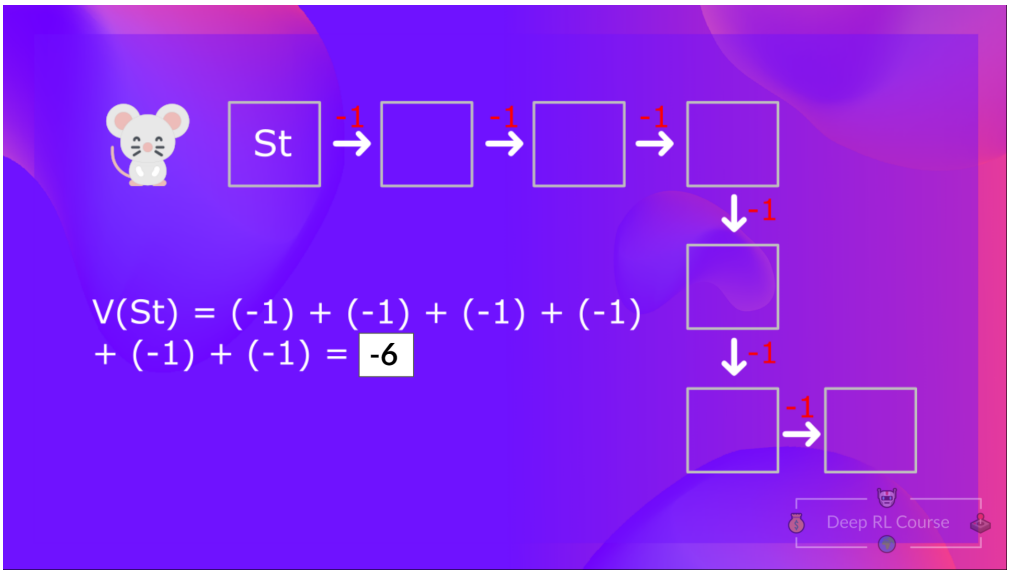


Figure 16: To calculate the value of State 1: the sum of rewards if the agent started in that state and then followed the greedy policy (taking actions that leads to the best states values) for all the time steps. [original figure]

Then, to calculate the $V(S_{t+1})$, we need to calculate the return starting at that state S_{t+1}.

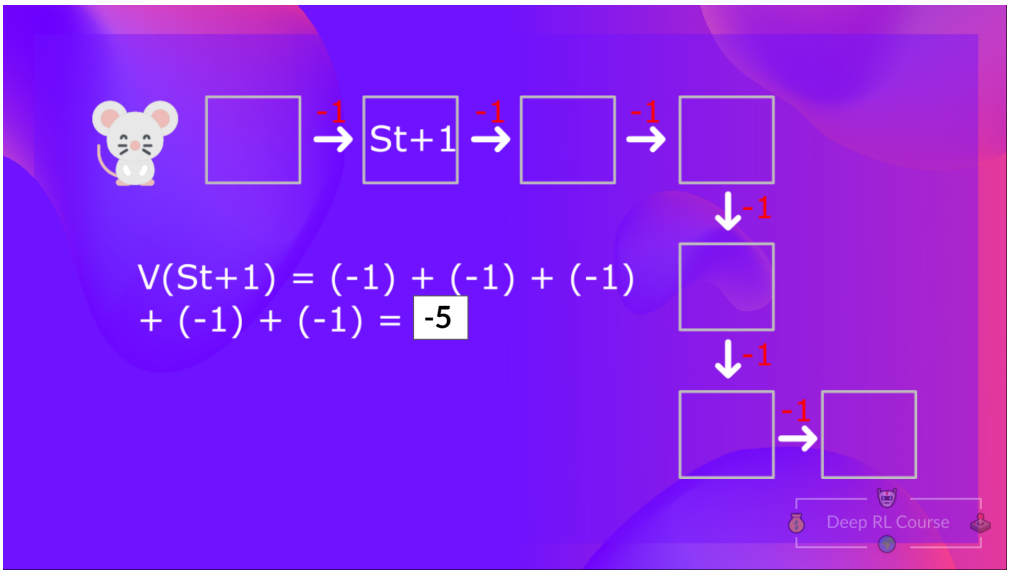


Figure 17: To calculate the value of State 2: the sum of rewards if the agent started in that state, and then followed the policy for all the time steps. [original figure]

So you may have noticed, we're repeating the computation of the value of different states, which can be tedious if you need to do it for each state value or state-action value.

Instead of calculating the expected return for each state or each state-action pair, **we can use the Bellman equation.** (hint: if you know what Dynamic Programming is, this is very similar! if you don't know what it is, no worries!)

The Bellman equation is a recursive equation that works like this: instead of starting for each state from the beginning and calculating the return, we can consider the value of any state as:

The immediate reward R_{t+1} + the discounted value of the state that follows ($\gamma * V(S_{t+1})$) .

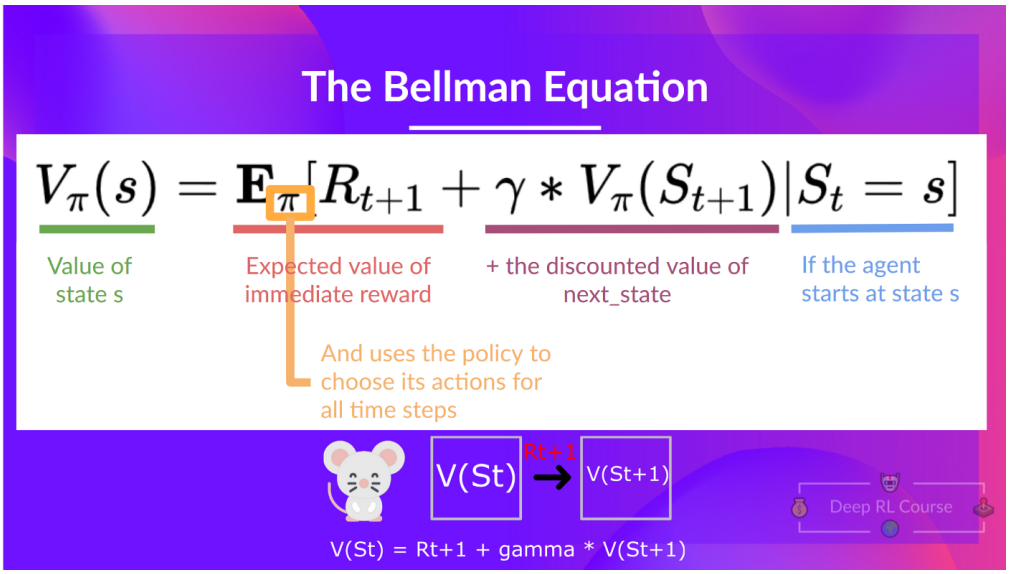


Figure 18: Bellman decomposition into immediate reward and discounted next-state value [original figure]

If we go back to our example, we can say that the value of State 1 is equal to the expected cumulative return if we start at that state.

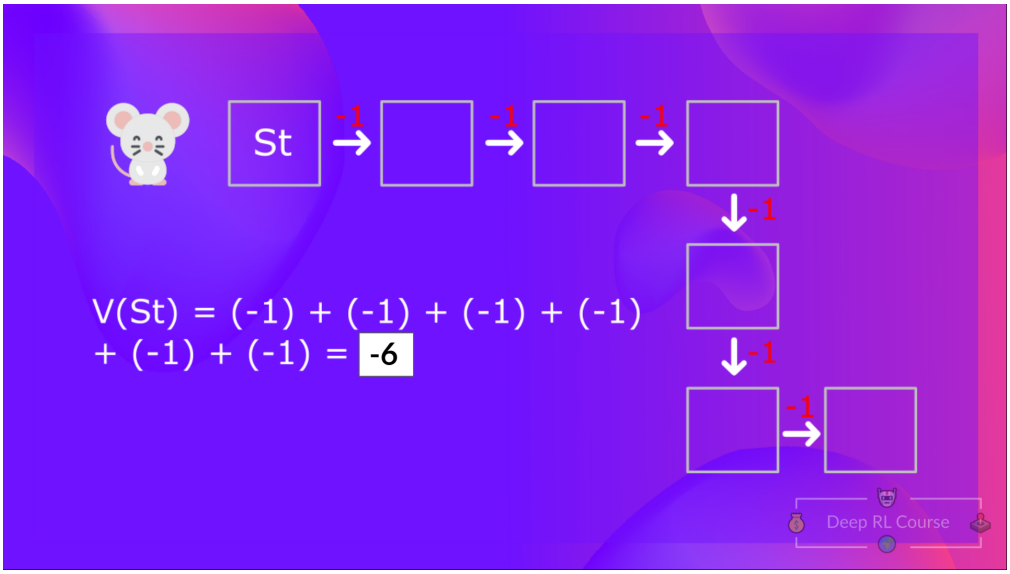


Figure 19: Return accumulated from the first state in a trajectory [original figure]

To calculate the value of State 1: the sum of rewards **if the agent started in that state 1** and then followed the **policy for all the time steps.**

This is equivalent to $V(S_t)$ = Immediate reward R_{t+1} + Discounted value of the next state $\gamma * V(S_{t+1})$

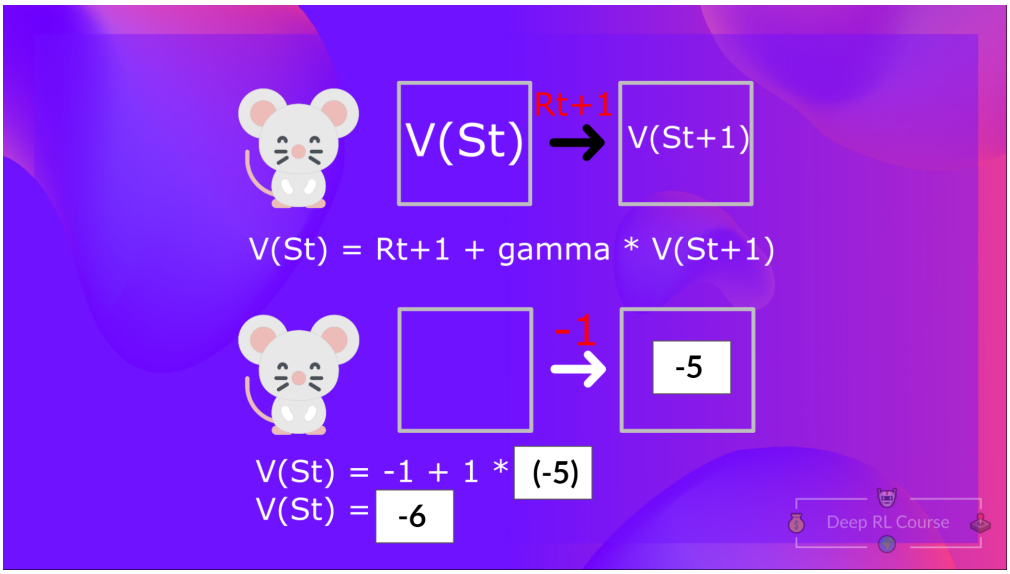


Figure 20: For simplification, here we don't discount so gamma = 1. [original figure]

In the interest of simplicity, here we don't discount, so gamma = 1. But you'll study an example with gamma = 0.99 in the Q-Learning section of this unit.

- The value of $V(S_{t+1})$ = Immediate reward R_{t+2} + Discounted value of the next state ($gamma * V(S_{t+2})$).
- And so on.

To recap, the idea of the Bellman equation is that instead of calculating each value as the sum of the expected return, **which is a long process**, we calculate the value as **the sum of immediate reward + the discounted value of the state that follows.**

Before going to the next section, think about the role of gamma in the Bellman equation. What happens if the value of gamma is very low (e.g. 0.1 or even 0)? What happens if the value is 1? What happens if the value is very high, such as a million?

5 Monte Carlo vs Temporal Difference Learning

The last thing we need to discuss before diving into Q-Learning is the two learning strategies.

Remember that an RL agent **learns by interacting with its environment.** The idea is that **given the experience and the received reward, the agent will update its value function or policy.**

Monte Carlo and Temporal Difference Learning are two different **strategies on how to train our value function or our policy function.** Both of them **use experience to solve the RL problem.**

On one hand, Monte Carlo uses **an entire episode of experience before learning.** On the other hand, Temporal Difference uses **only a step ($S_t, A_t, R_{t+1}, S_{t+1}$) to learn.**

We'll explain both of them **using a value-based method example.**

5.1 Monte Carlo: learning at the end of the episode

Monte Carlo waits until the end of the episode, calculates G_t (return) and uses it as **a target for updating $V(S_t)$.**

So it requires a **complete episode of interaction before updating our value function.**

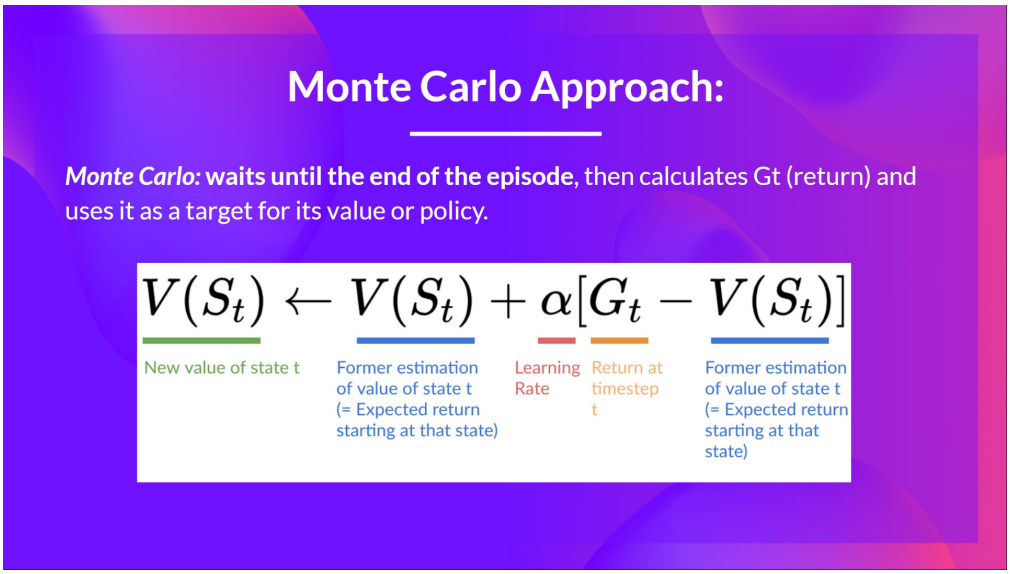


Figure 21: Monte Carlo value estimation from complete episode returns [original figure]

If we take an example:

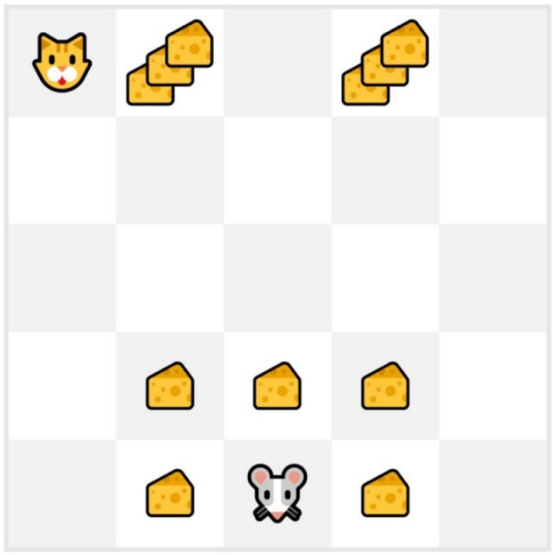


Figure 22: Monte Carlo learning: worked example (stage 2) [original figure]

- We always start the episode **at the same starting point.**
- **The agent takes actions using the policy**. For instance, using an Epsilon Greedy Strategy, a policy that alternates between exploration (random actions) and exploitation.
- We get **the reward and the next state.**
- We terminate the episode if the cat eats the mouse or if the mouse moves > 10 steps.
- At the end of the episode, **we have a list of State, Actions, Rewards, and Next States tuples**

For instance [[State tile 3 bottom, Go Left, +1, State tile 2 bottom], [State tile 2 bottom, Go Left, +0, State tile 1 bottom]...]

- **The agent will sum the total rewards** G_t (to see how well it did).
- It will then **update** $V(s_t)$ **based on the formula**

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

New value of state t

Former estimation of value of state t (= Expected return starting at that state)

Learning Rate

Return at timestep t

Former estimation of value of state t (= Expected return starting at that state)

Figure 23: Monte Carlo learning: worked example (stage 3) [original figure]

- Then **start a new game with this new knowledge**

By running more and more episodes, **the agent will learn to play better and better.**

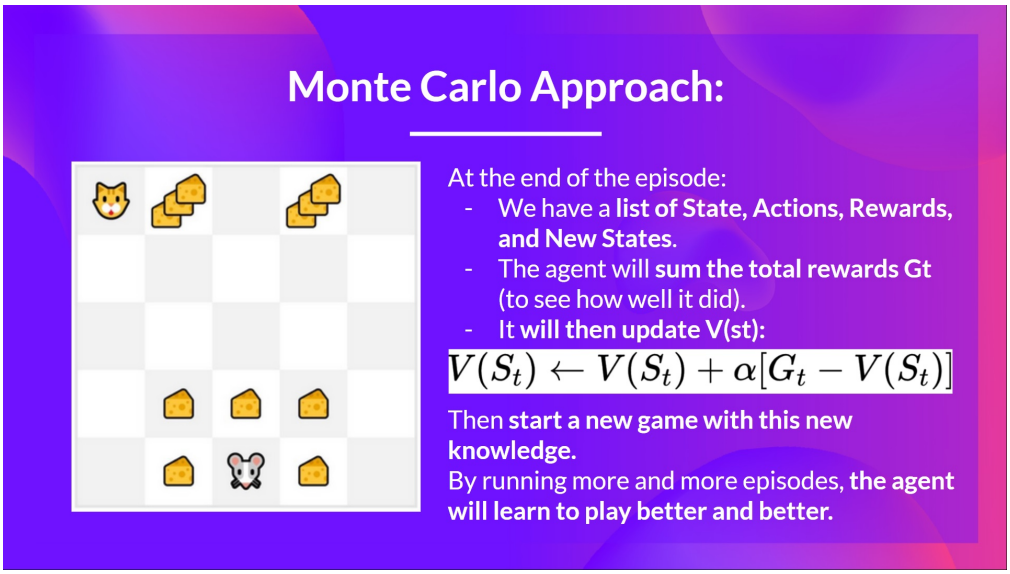


Figure 24: Monte Carlo learning: worked example (stage 3) [original figure]

For instance, if we train a state-value function using Monte Carlo:

- We initialize our value function **so that it returns 0 value for each state**
- Our learning rate (lr) is 0.1 and our discount rate is 1 (= no discount)
- Our mouse **explores the environment and takes random actions**

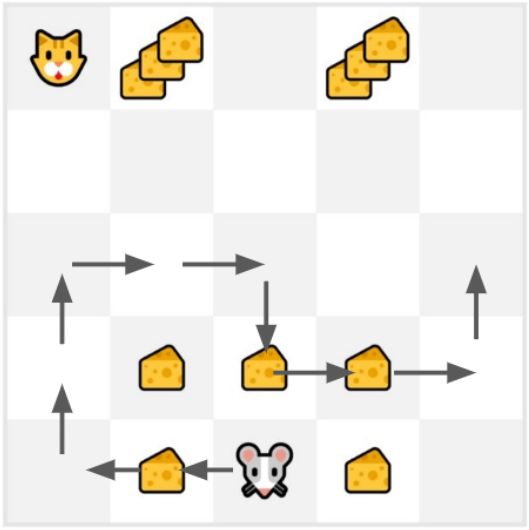


Figure 25: Monte Carlo learning: worked example (stage 4) [original figure]

- The mouse made more than 10 steps, so the episode ends .

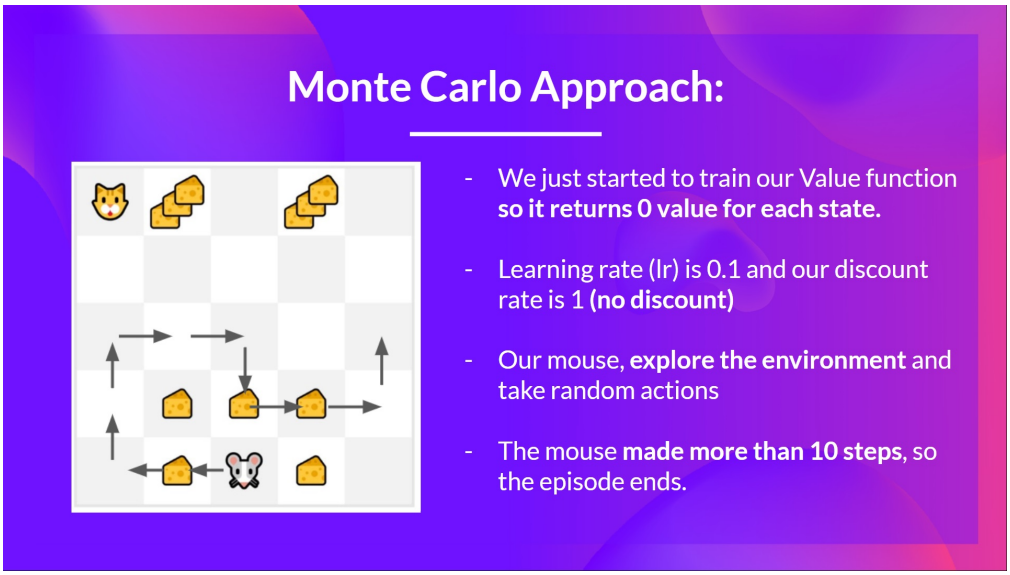


Figure 26: Monte Carlo learning: worked example (stage 4) [original figure]

- We have a list of state, action, rewards, next_state, **we need to calculate the return** $Gt = 0$

$G_t = R_{t+1} + R_{t+2} + R_{t+3}$... (for simplicity, we don't discount the rewards)
$G_0 = R_1 + R_2 + R_3 \dots$
$G_0 = 1 + 0 + 0 + 0 + 0 + 0 + 1 + 1 + 0 + 0$
$G_0 = 3$

- We can now compute the **new** $V(S_0)$:

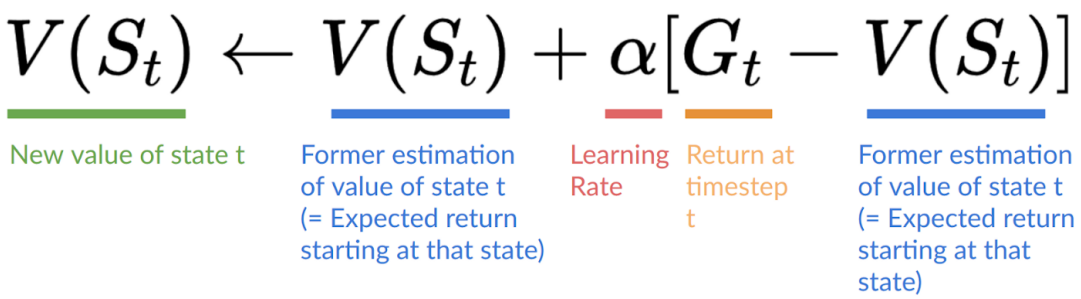


Figure 27: Monte Carlo learning: worked example (stage 5) [original figure]

$V(S_0) = V(S_0) + lr * [G_0 - V(S_0)]$
$V(S_0) = 0 + 0.1 * [3 - 0]$
$V(S_0) = 0.3$

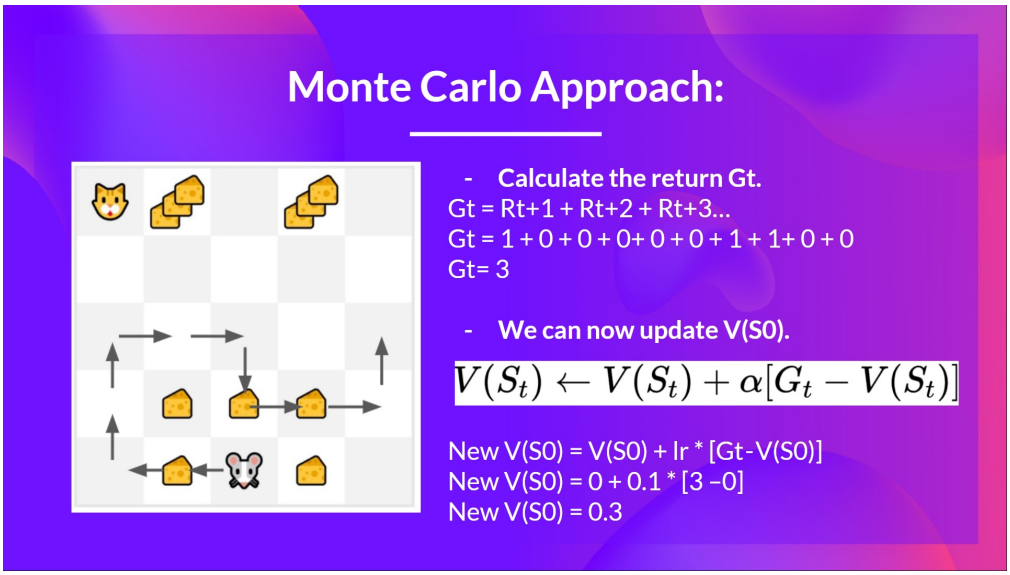


Figure 28: Monte Carlo learning: worked example (stage 5) [original figure]

5.2 Temporal Difference Learning: learning at each step

Temporal Difference, on the other hand, waits for only one interaction (one step) S_{t+1} to form a TD target and update $V(S_t)$ using R_{t+1} and $\gamma * V(S_{t+1})$.

The idea with **TD is to update the** $V(S_t)$ **at each step.**

But because we didn't experience an entire episode, we don't have G_t (expected return). Instead, **we estimate** G_t **by adding** R_{t+1} **and the discounted value of the next state.**

This is called bootstrapping. It's called this **because TD bases its update in part on an existing estimate** $V(S_{t+1})$ **and not a complete sample** G_t**.**

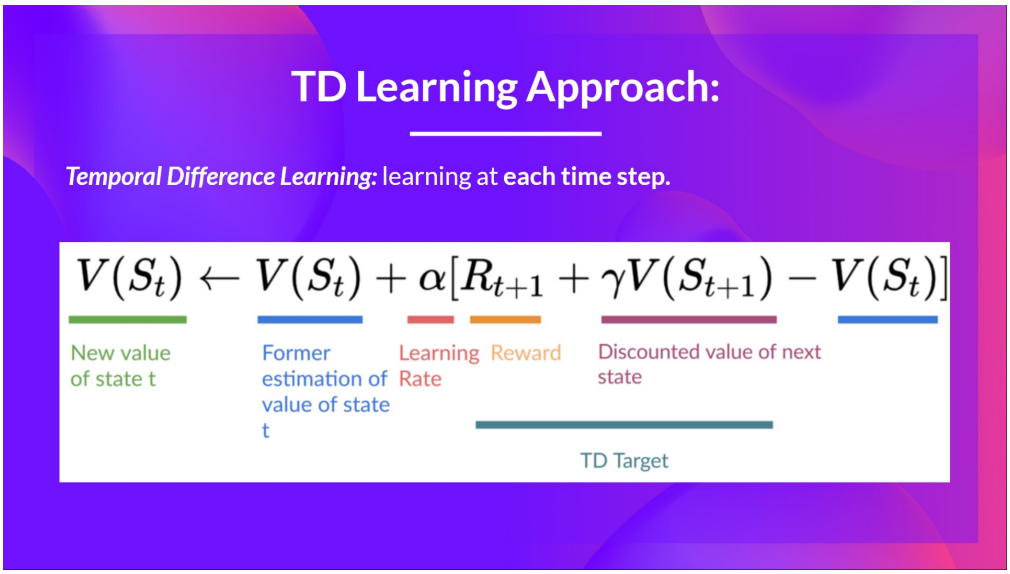


Figure 29: Temporal Difference learning: worked example (stage 1) [original figure]

This method is called TD(0) or **one-step TD (update the value function after any individual step).**



Figure 30: Temporal Difference learning: worked example (stage 1) [original figure]

If we take the same example,

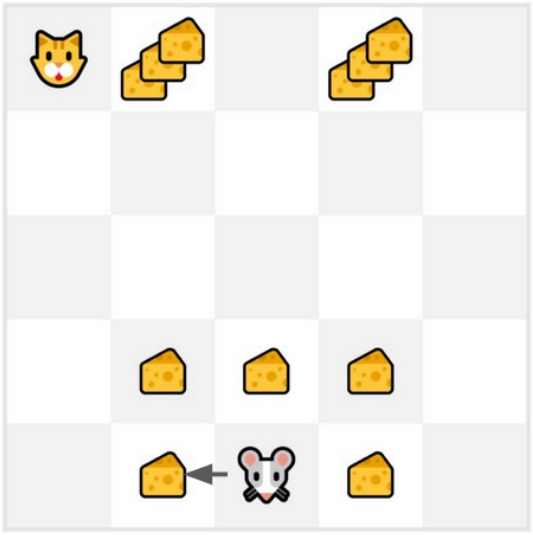


Figure 31: Temporal Difference learning: worked example (stage 2) [original figure]

- We initialize our value function so that it returns 0 value for each state.
- Our learning rate (lr) is 0.1, and our discount rate is 1 (no discount).
- Our mouse begins to explore the environment and takes a random action: **going to the left**
- It gets a reward $R_{t+1} = 1$ since **it eats a piece of cheese**

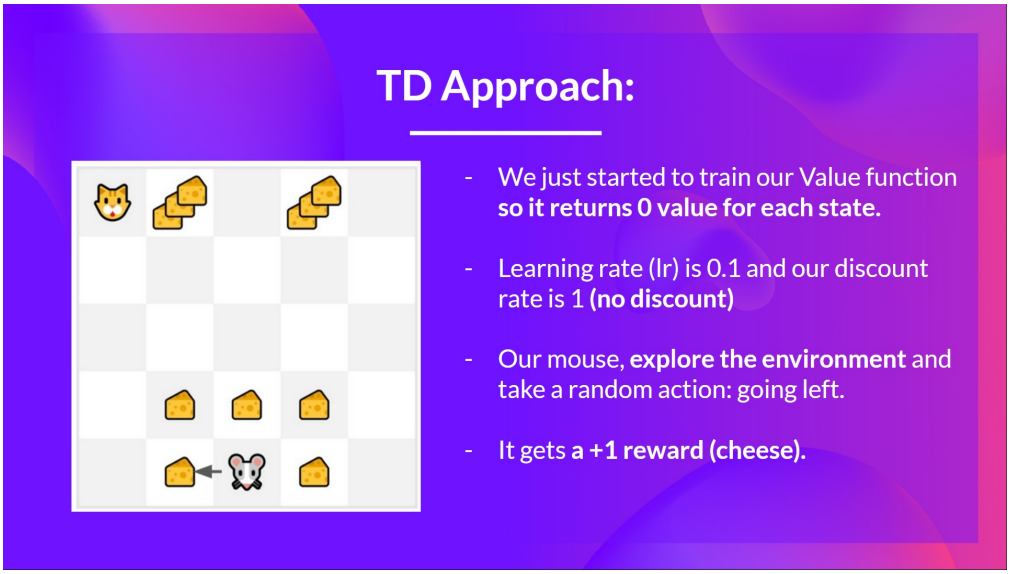


Figure 32: Temporal Difference learning: worked example (stage 2) [original figure]

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

New value of state t

Former estimation of value of state t

Learning Rate

Reward

Discounted value of next state

TD Target

Figure 33: Temporal Difference learning: worked example (stage 3) [original figure]

We can now update $V(S_0)$:

New $V(S_0) = V(S_0) + lr * [R_1 + \gamma * V(S_1) - V(S_0)]$

New $V(S_0) = 0 + 0.1 * [1 + 1 * 0 - 0]$

New $V(S_0) = 0.1$

So we just updated our value function for State 0.

Now we **continue to interact with this environment with our updated value function.**

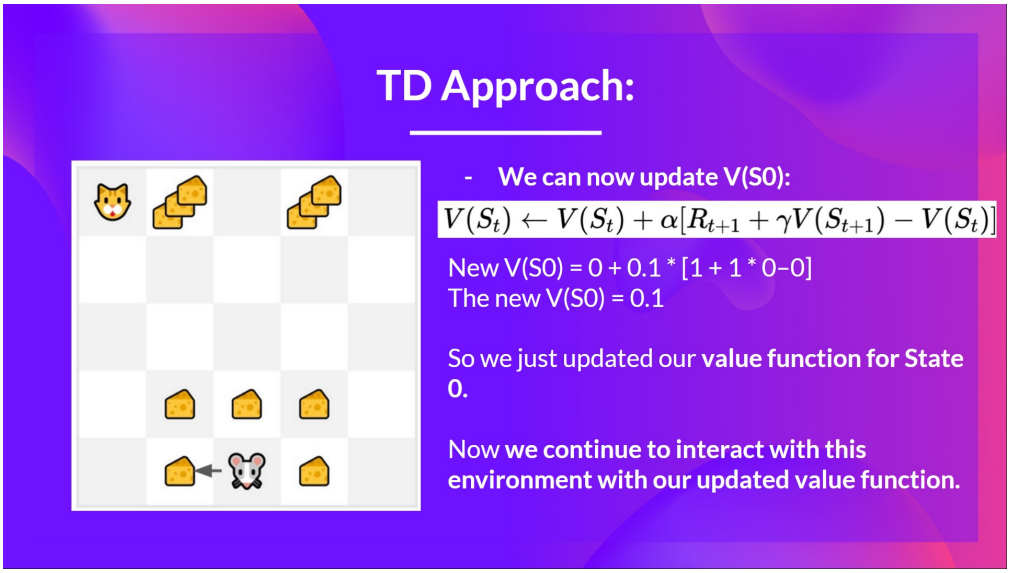


Figure 34: Temporal Difference learning: worked example (stage 3) [original figure]

To summarize:

- With *Monte Carlo*, we update the value function from a complete episode, and so we **use the actual accurate discounted return of this episode.**
- With *TD Learning*, we update the value function from a step, and we replace G_t, which we don't know, with **an estimated return called the TD target.**

Monte Carlo: $V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$

TD Learning: $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

Figure 35: Monte Carlo versus temporal-difference learning: comparison [original figure]

6 Mid-way Recap

Before diving into Q-Learning, let's summarize what we've just learned.

We have two types of value-based functions:

- State-value function: outputs the expected return if **the agent starts at a given state and acts according to the policy forever after.**
- Action-value function: outputs the expected return if **the agent starts in a given state, takes a given action at that state** and then acts accordingly to the policy forever after.
- In value-based methods, rather than learning the policy, **we define the policy by hand** and we learn a value function. If we have an optimal value function, we **will have an optimal policy.**

There are two types of methods to update the value function:

- With *the Monte Carlo method*, we update the value function from a complete episode, and so we **use the actual discounted return of this episode.**

- With *the TD Learning method,* we update the value function from a step, replacing the unknown G_t with **an estimated return called the TD target.**

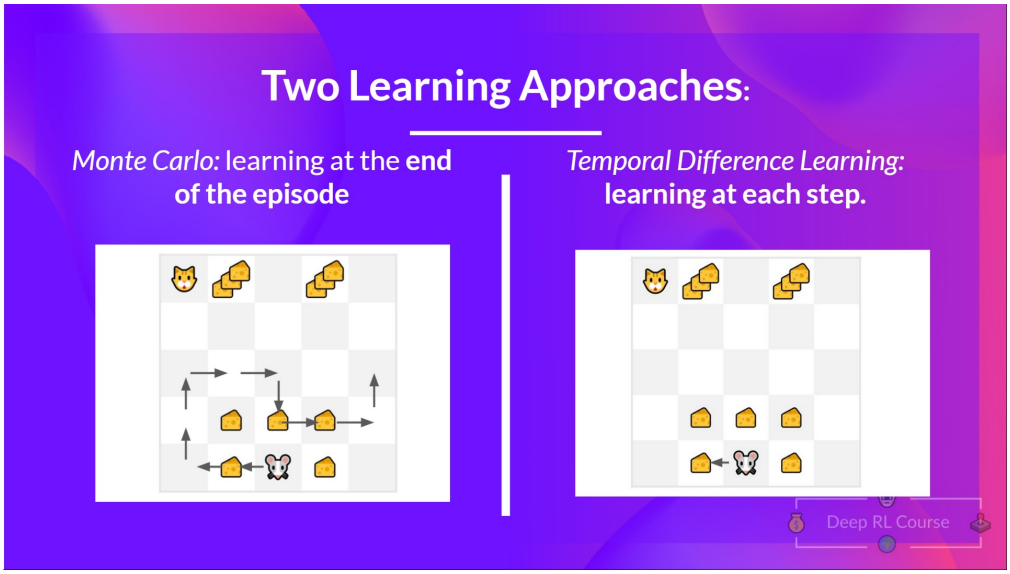


Figure 36: Summary [original figure]

7 Mid-way Quiz

The best way to learn and to avoid the illusion of competence **is to test yourself.** This will help you to find **where you need to reinforce your knowledge**.

Q1: What are the two main approaches to find optimal policy?

Select all applicable answers.

- **A.** Policy-based methods
- **B.** Random-based methods
- **C.** Value-based methods
- **D.** Evolution-strategies methods

Q2: What is the Bellman Equation?

Q3: Define each part of the Bellman Equation

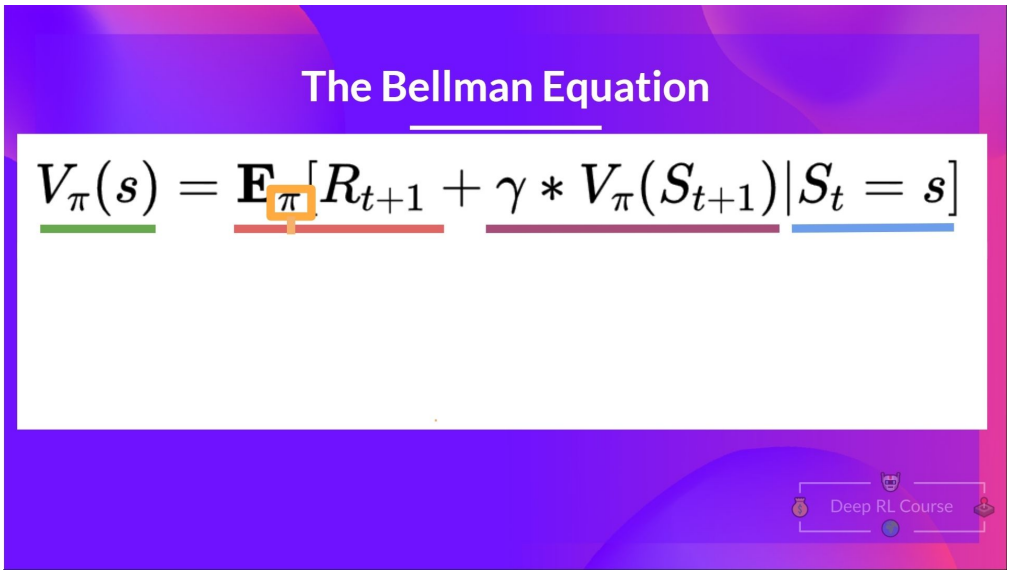


Figure 37: Bellman equation quiz [original figure]

Q4: What is the difference between Monte Carlo and Temporal Difference learning methods?

Select all applicable answers.

- **A.** With Monte Carlo methods, we update the value function from a complete episode
- **B.** With Monte Carlo methods, we update the value function from a step
- **C.** With TD learning methods, we update the value function from a complete episode
- **D.** With TD learning methods, we update the value function from a step

Q5: Define each part of Temporal Difference learning formula

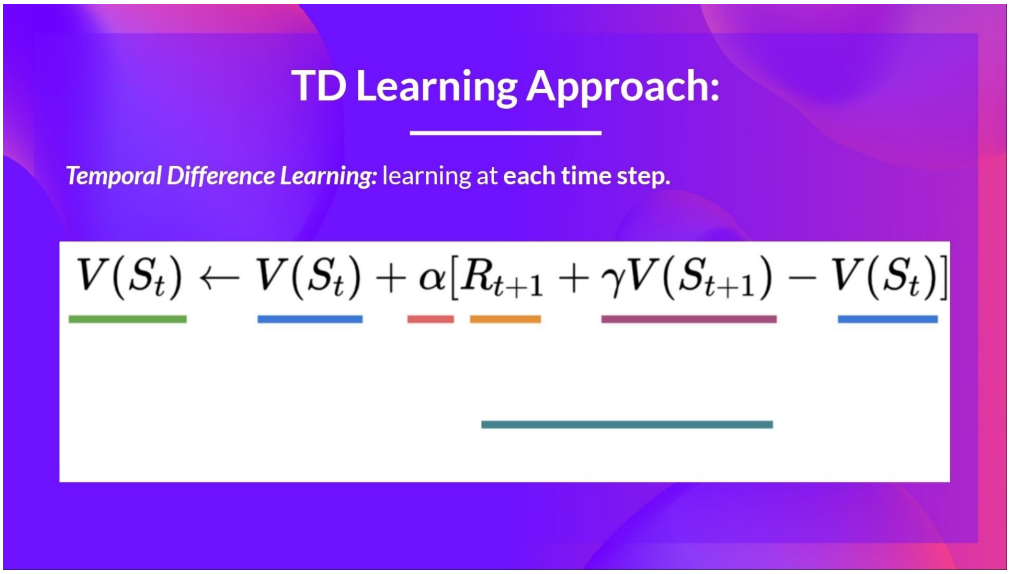


Figure 38: TD Learning exercise [original figure]

Q6: Define each part of Monte Carlo learning formula

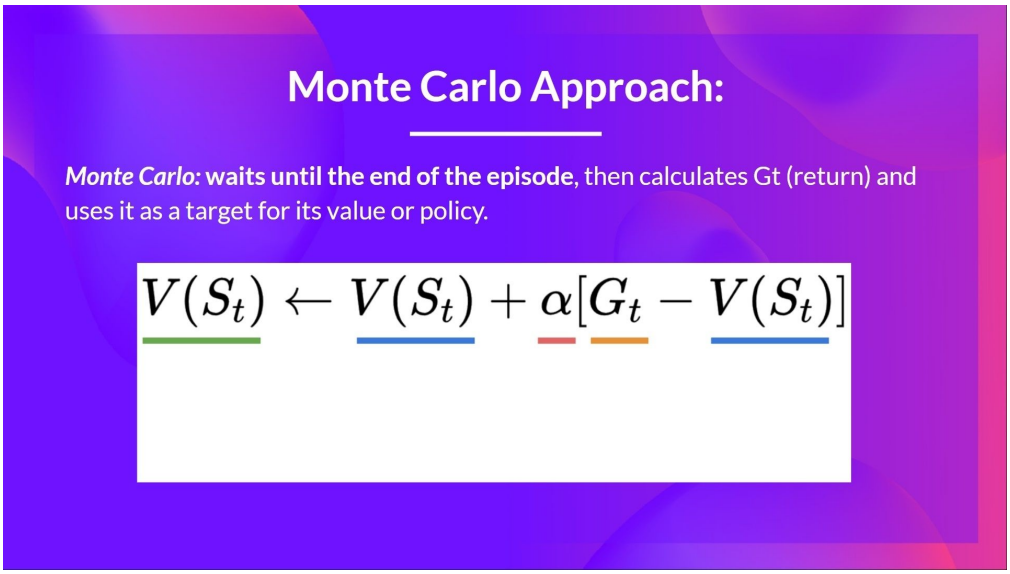


Figure 39: MC Learning exercise [original figure]

7.1 Answer Key and Explanations

Q1: What are the two main approaches to find optimal policy?

Correct choice(s): A, C

- **Option A (correct):** With Policy-Based methods, we train the policy directly to learn which action to take given a state.
- **Option C (correct):** With value-based methods, we train a value function to learn which state is more valuable and use this value function to take the action that leads to it.

Q2: What is the Bellman Equation?

The Bellman equation is a recursive equation that works like this: instead of starting for each state from the beginning and calculating the return, we can consider the value of any state as:

Rt+1 + gamma * V(St+1)

The immediate reward + the discounted value of the state that follows

Q3: Define each part of the Bellman Equation

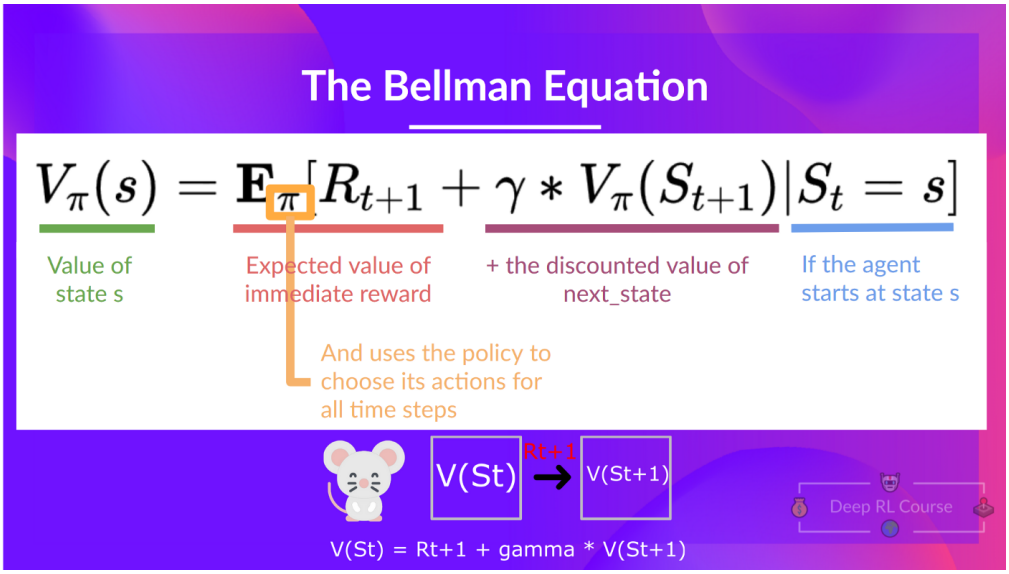


Figure 40: Bellman decomposition into immediate reward and discounted next-state value [original figure]

Q4: What is the difference between Monte Carlo and Temporal Difference learning methods?

Correct choice(s): A, D

Q5: Define each part of Temporal Difference learning formula

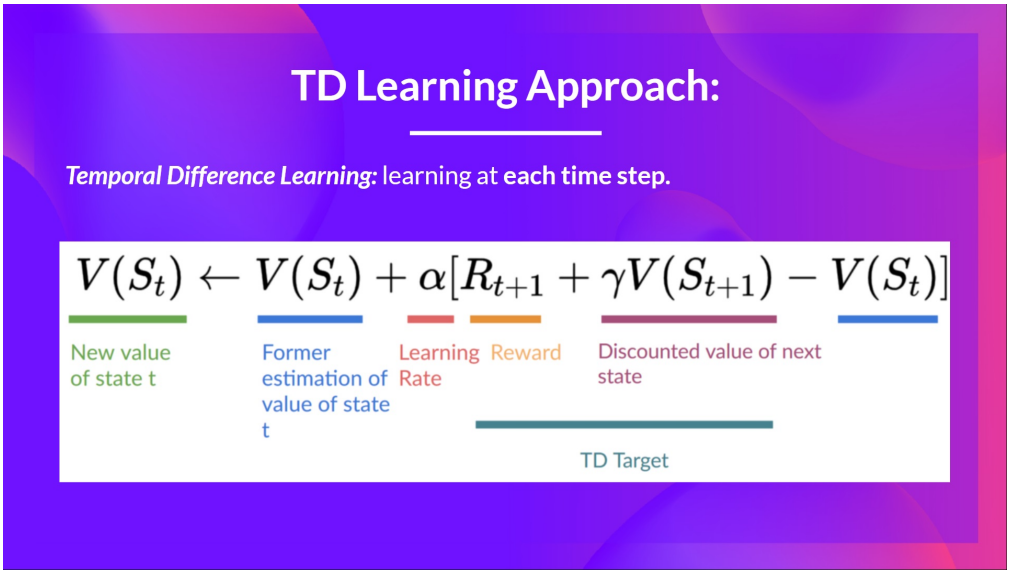


Figure 41: Temporal Difference learning: worked example (stage 1) [original figure]

Q6: Define each part of Monte Carlo learning formula

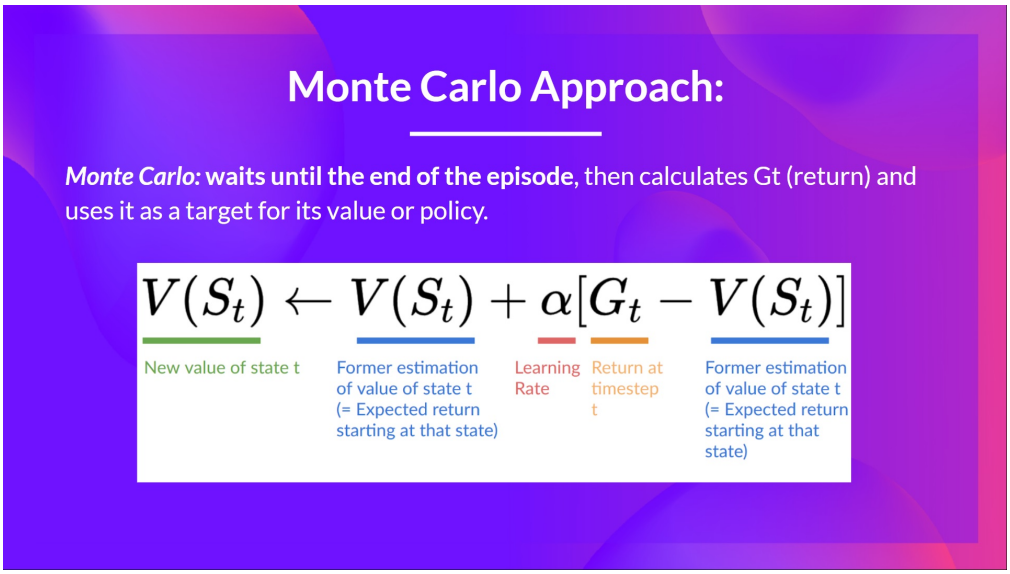


Figure 42: Monte Carlo value estimation from complete episode returns [original figure]

Congrats on finishing this Quiz , if you missed some elements, take time to read again the previous sections to reinforce () your knowledge.

8 Introducing Q-Learning

8.1 What is Q-Learning?

Q-Learning is an **off-policy value-based method that uses a TD approach to train its action-value function:**

- *Off-policy*: we'll talk about that at the end of this unit.
- *Value-based method*: finds the optimal policy indirectly by training a value or action-value function that will tell us **the value of each state or each state-action pair.**

- *TD approach:* **updates its action-value function at each step instead of at the end of the episode.**

Q-Learning is the algorithm we use to train our Q-function, an **action-value function** that determines the value of being at a particular state and taking a specific action at that state.

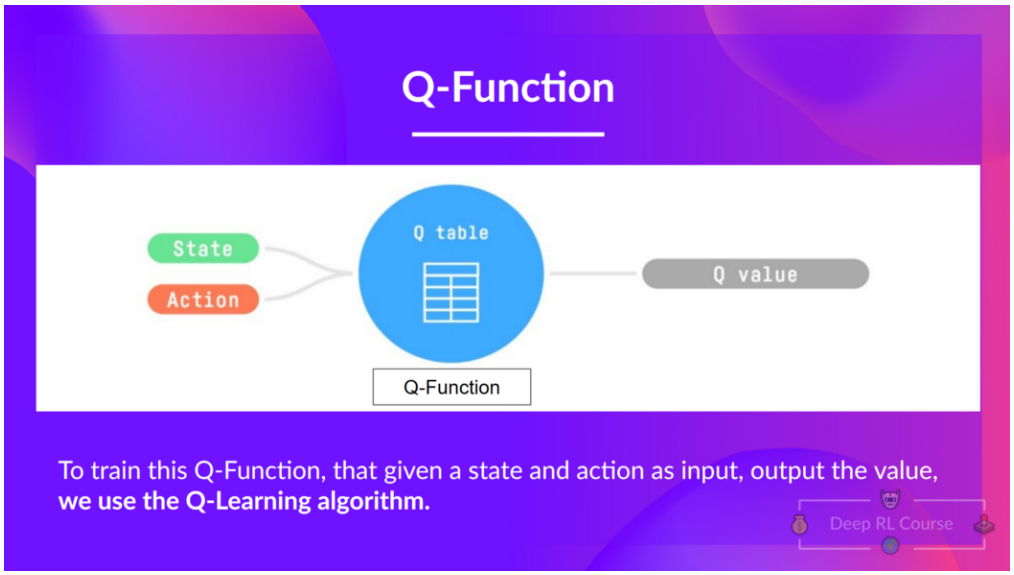


Figure 43: Given a state and action, our Q Function outputs a state-action value (also called Q-value) [original figure]

The **Q comes from "the Quality" (the value) of that action at that state.**

Let's recap the difference between value and reward:

- The *value of a state*, or a *state-action pair* is the expected cumulative reward our agent gets if it starts at this state (or state-action pair) and then acts accordingly to its policy.
- The *reward* is the **feedback it gets from the environment** after performing an action at a state.

Internally, our Q-function is encoded by **a Q-table, a table where each cell corresponds to a state-action pair value.** Think of this Q-table as **the memory or cheat sheet of our Q-function.**

Let's go through an example of a maze.



Figure 44: Q-table illustrated on a mouse-and-cheese maze (Maze-1) [original figure]

The Q-table is initialized. That's why all values are = 0. This table **contains, for each state and action, the corresponding state-action values.** For this simple example, the state is only defined by the position of the mouse. Therefore, we have 2*3 rows in our Q-table, one row for each possible position of the mouse. In more complex scenarios, the state could contain more information than the position of the actor.

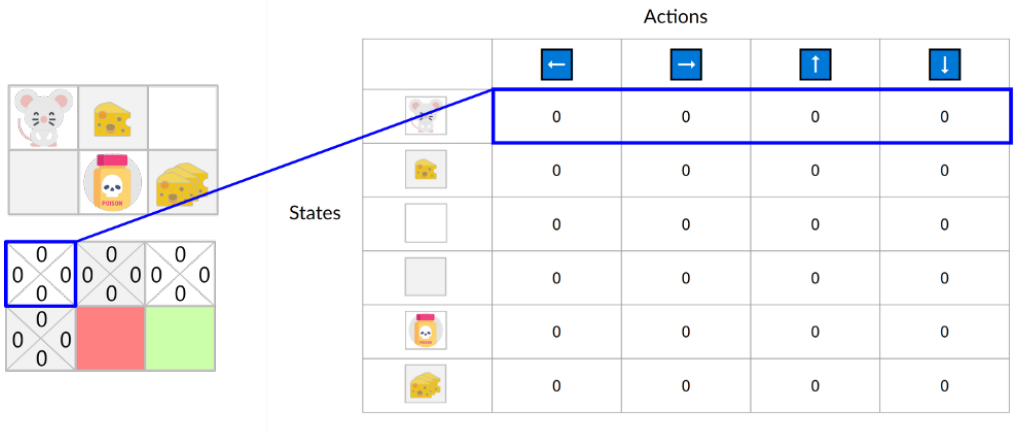


Figure 45: Q-table illustrated on a mouse-and-cheese maze (Maze-2) [original figure]

Here we see that the **state-action value of the initial state and going up is 0:**

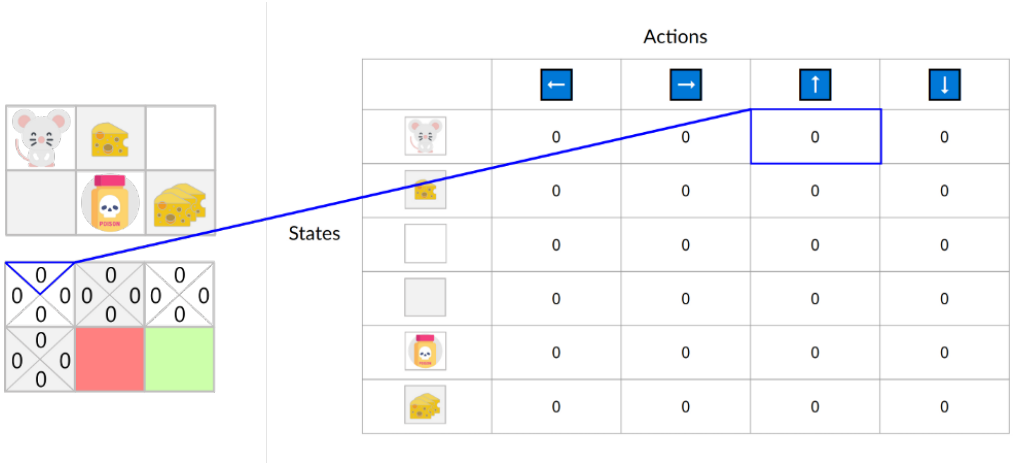


Figure 46: Q-table illustrated on a mouse-and-cheese maze (Maze-3) [original figure]

So: the Q-function uses a Q-table **that has the value of each state-action pair.** Given a state and action, **our Q-function will search inside its Q-table to output the value.**

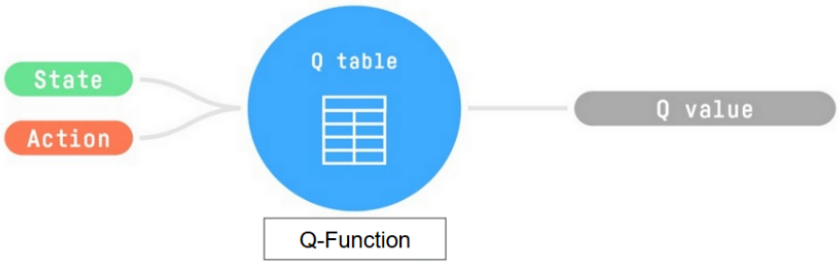


Figure 47: Q-table lookup: obtaining the value of a state-action pair [original figure]

If we recap, *Q-Learning* **is the RL algorithm that:**

- Trains a *Q-function* (an **action-value function**), which internally is a **Q-table that contains all the state-action pair values.**

- Given a state and action, our Q-function **will search its Q-table for the corresponding value.**
- When the training is done, **we have an optimal Q-function, which means we have optimal Q-table.**
- And if we **have an optimal Q-function**, we **have an optimal policy** since we **know the best action to take at each state.**

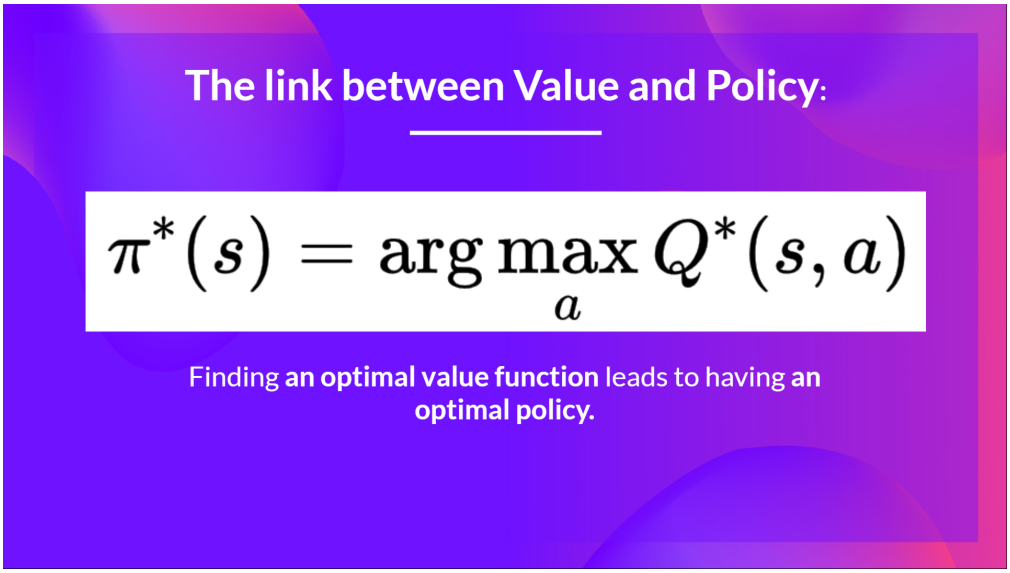


Figure 48: Recovering an optimal policy from the optimal action-value function [original figure]

In the beginning, **our Q-table is useless since it gives arbitrary values for each state-action pair** (most of the time, we initialize the Q-table to 0). As the agent **explores the environment and we update the Q-table, it will give us a better and better approximation** to the optimal policy.



Figure 49: We see here that with the training, our Q-table is better since, thanks to it, we can know the value of each state-action pair. [original figure]

Now that we understand what Q-Learning, Q-functions, and Q-tables are, **let's dive deeper into the Q-Learning algorithm**.

8.2 The Q-Learning algorithm

This is the Q-Learning pseudocode; let's study each part and **see how it works with a simple example before implementing it.** Don't be intimidated by it, it's simpler than it looks! We'll go over each step.

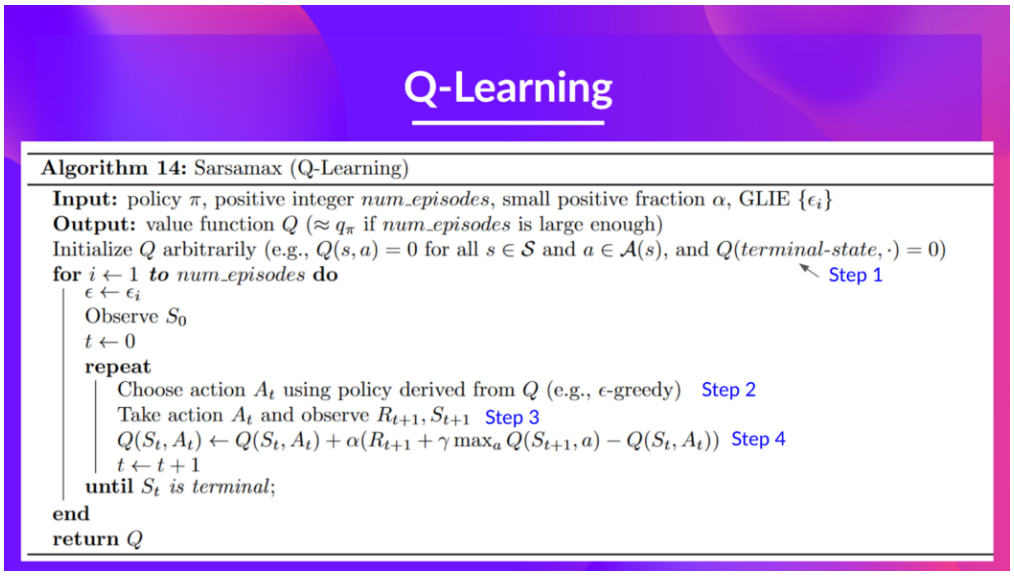


Figure 50: Q-learning: reference algorithm and update pseudocode [original figure]

Step 1: We initialize the Q-table

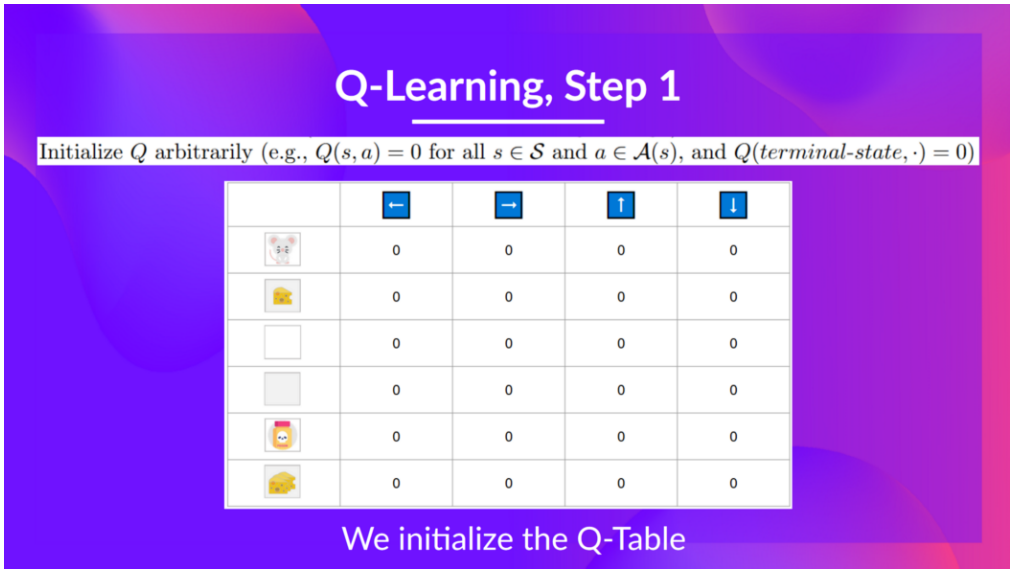


Figure 51: Q-learning: choose action using epsilon-greedy policy [original figure]

We need to initialize the Q-table for each state-action pair. **Most of the time, we initialize with values of 0.**

Step 2: Choose an action using the epsilon-greedy strategy

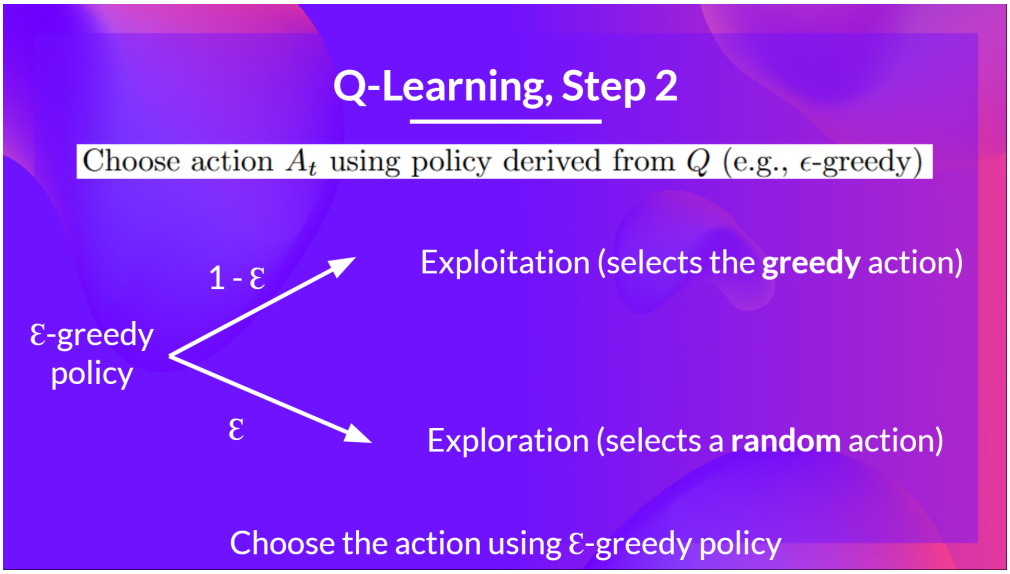


Figure 52: Epsilon-greedy strategy: exploration and exploitation [original figure]

The epsilon-greedy strategy is a policy that handles the exploration/exploitation trade-off.

The idea is that, with an initial value of $\varepsilon = 1.0$:

- *With probability 1 — ε :* we do **exploitation** (aka our agent selects the action with the highest state-action pair value).
- With probability ε: **we do exploration** (trying random action).

At the beginning of the training, **the probability of doing exploration will be huge since ε is very high, so most of the time, we'll explore.** But as the training goes on, and consequently our **Q-table gets better and better in its estimations, we progressively reduce the epsilon value** since we will need less and less exploration and more exploitation.

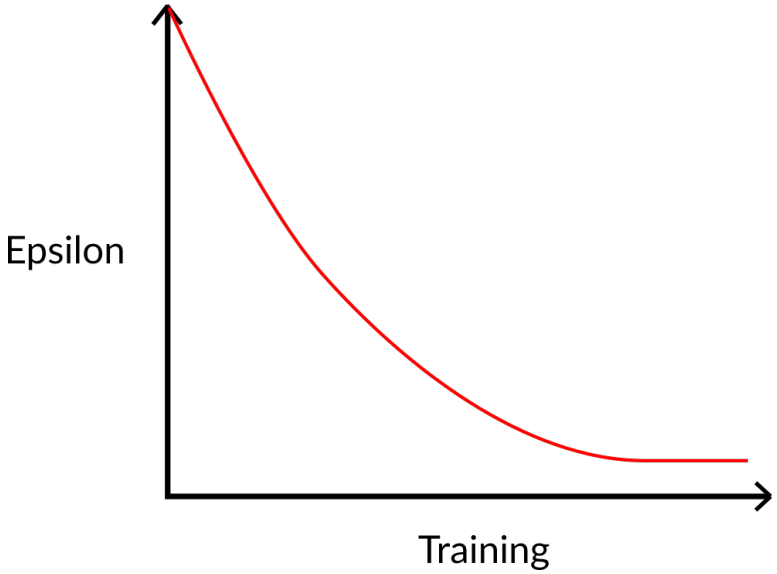


Figure 53: Q-learning: perform action and observe transition [original figure]

Step 3: Perform action At, get reward Rt+1 and next state St+1

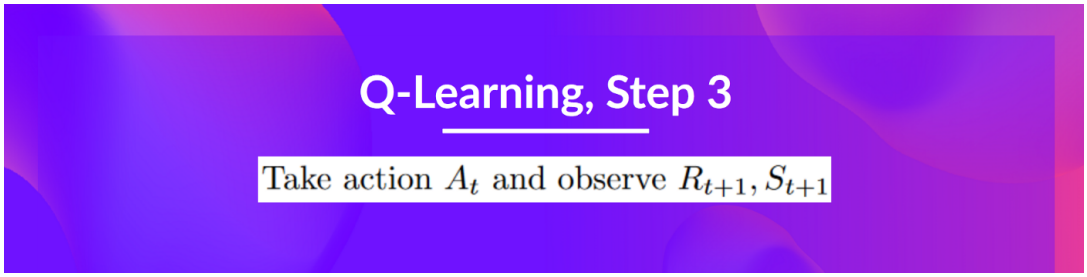


Figure 54: Q-learning: update the action value using a TD target [original figure]

Step 4: Update Q(St, At)

Remember that in TD Learning, we update our policy or value function (depending on the RL method we choose) **after one step of the interaction.**

To produce our TD target, **we used the immediate reward R_{t+1} plus the discounted value of the next state**, computed by finding the action that maximizes the current Q-function at the next state. (We call that bootstrap).

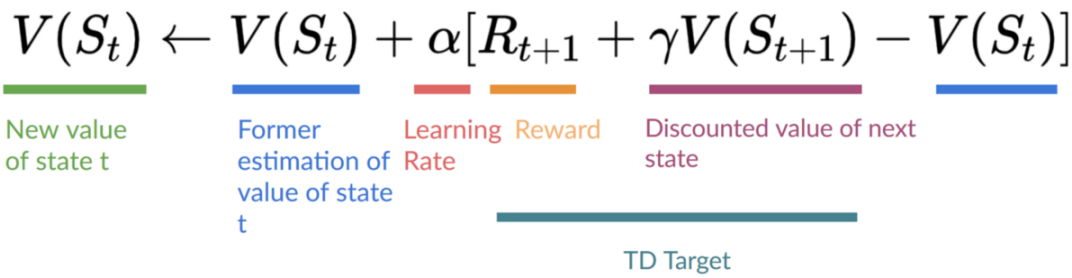


Figure 55: Q-learning: repeat updates over multiple episodes [original figure]

Therefore, our $Q(S_t, A_t)$ **update formula goes like this:**

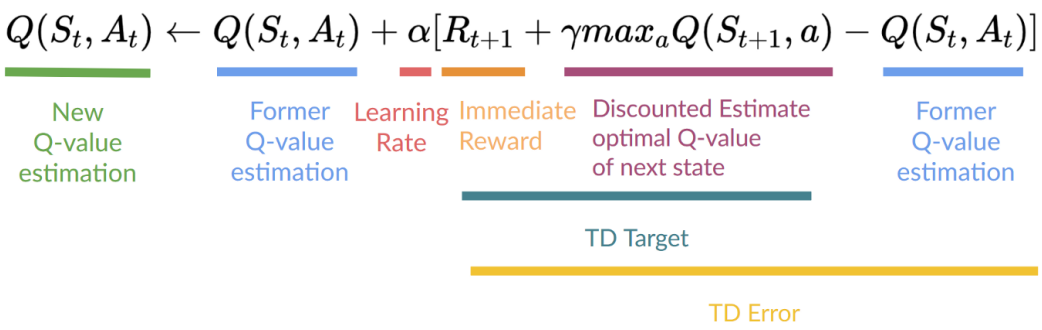


Figure 56: Q-learning: complete algorithm overview [original figure]

This means that to update our $Q(S_t, A_t)$:

- We need $S_t, A_t, R_{t+1}, S_{t+1}$.
- To update our Q-value at a given state-action pair, we use the TD target.

How do we form the TD target?

1. We obtain the reward R_{t+1} after taking the action A_t.
2. To get the **best state-action pair value** for the next state, we use a greedy policy to select the next best action. Note that this is not an epsilon-greedy policy, this will always take the action with the highest state-action value.

Then when the update of this Q-value is done, we start in a new state and select our action **using a epsilon-greedy policy again.**

This is why we say that Q Learning is an off-policy algorithm.

8.3 Off-policy vs On-policy

The difference is subtle:

- *Off-policy*: using **a different policy for acting (inference) and updating (training).**

For instance, with Q-Learning, the epsilon-greedy policy (acting policy), is different from the greedy policy that is **used to select the best next-state action value to update our Q-value (updating policy).**

Choose action A_t using policy derived from Q (e.g., ϵ-greedy)

Figure 57: Off-on policy [original figure]

Is different from the policy we use during the training part:

$$\gamma \max_a Q(S_{t+1}, a)$$

Figure 58: Off-on policy [original figure]

- *On-policy:* using the **same policy for acting and updating.**

For instance, with Sarsa, another value-based algorithm, **the epsilon-greedy policy selects the next state-action pair, not a greedy policy.**

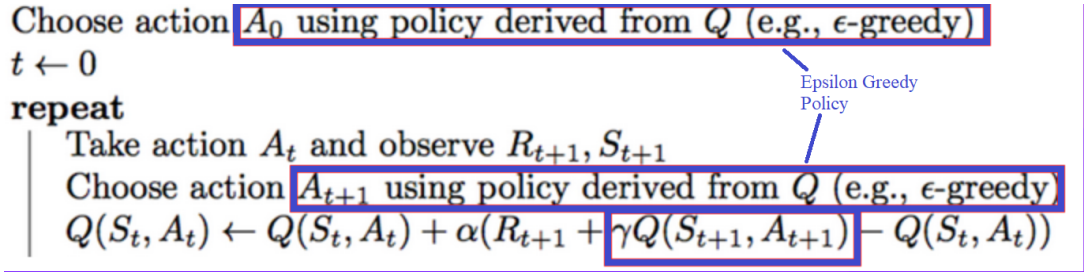


Figure 59: Off-on policy [original figure]

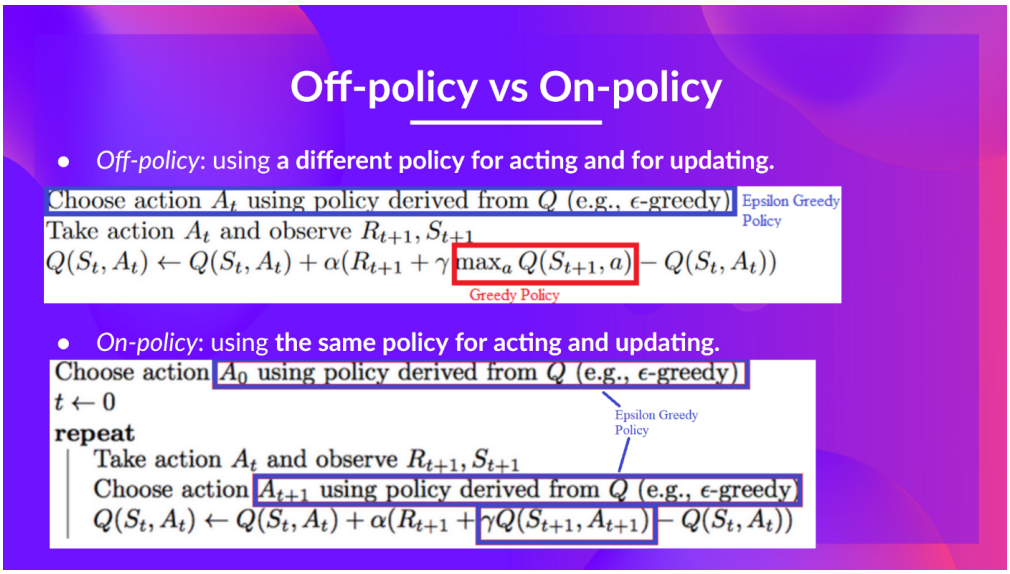


Figure 60: Comparison of off-policy and on-policy methods [original figure]

9 A Q-Learning example

To better understand Q-Learning, let's take a simple example:



Figure 61: The mouse-and-cheese maze with reward and termination rules [original figure]

- You're a mouse in this tiny maze. You always **start at the same starting point.**
- The goal is **to eat the big pile of cheese at the bottom right-hand corner** and avoid the poison. After all, who doesn't like cheese?
- The episode ends if we eat the poison, **eat the big pile of cheese**, or if we take more than five steps.
- The learning rate is 0.1
- The discount rate (gamma) is 0.99

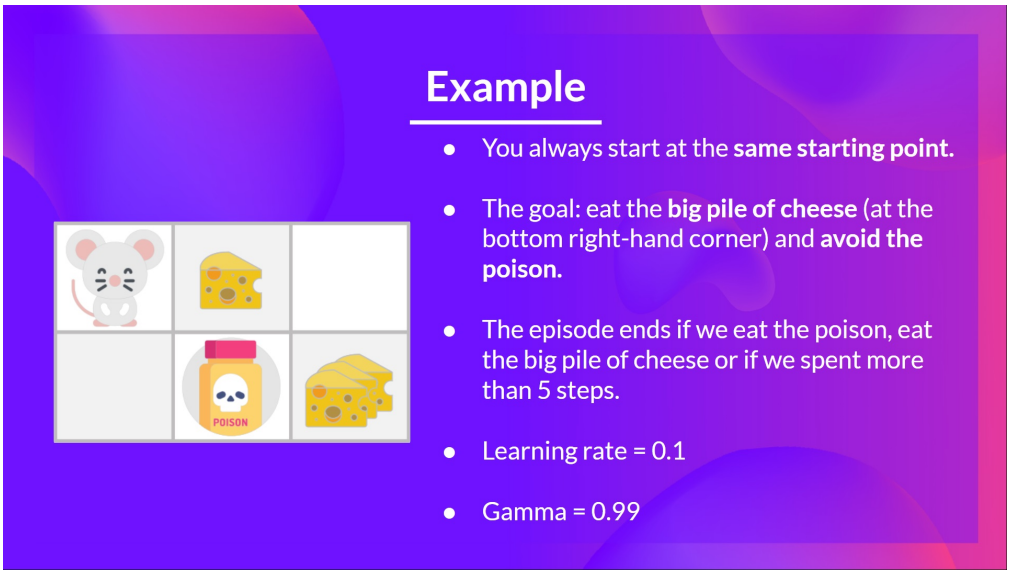


Figure 62: Q-learning example: transition and Q-table update (step 1) [original figure]

The reward function goes like this:

- **+0:** Going to a state with no cheese in it.
- **+1:** Going to a state with a small cheese in it.
- **+10:** Going to the state with the big pile of cheese.
- **-10:** Going to the state with the poison and thus dying.
- **+0** If we take more than five steps.

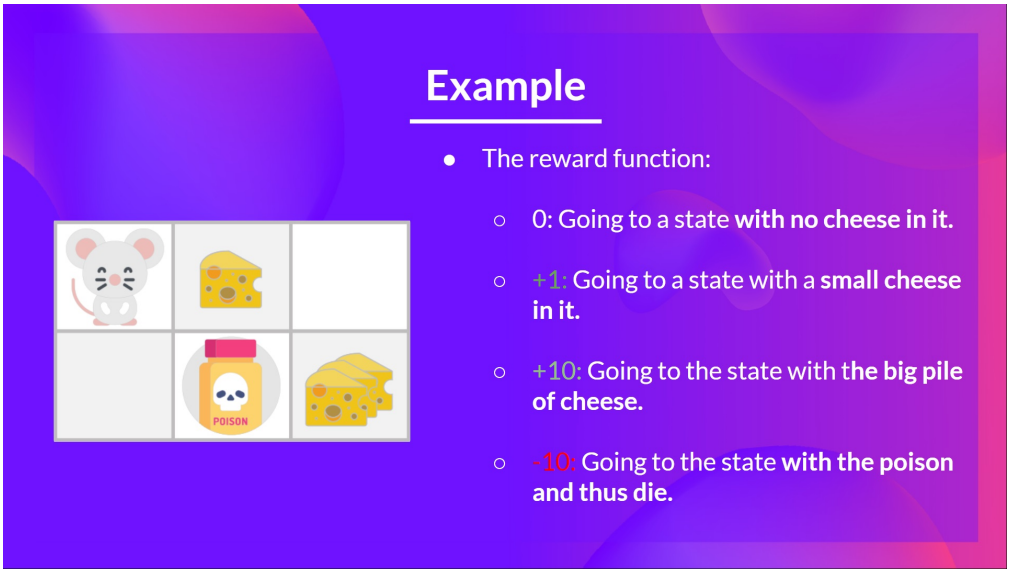


Figure 63: Q-learning example: transition and Q-table update (step 2) [original figure]

To train our agent to have an optimal policy (so a policy that goes right, right, down), **we will use the Q-Learning algorithm**.

9.1 Step 1: Initialize the Q-table



Figure 64: Maze-Example [original figure]

So, for now, **our Q-table is useless**; we need **to train our Q-function using the Q-Learning algorithm.**

Let's do it for 2 training timesteps:

Training timestep 1:

9.2 Step 2: Choose an action using the Epsilon Greedy Strategy

Because epsilon is big (= 1.0), I take a random action. In this case, I go right.

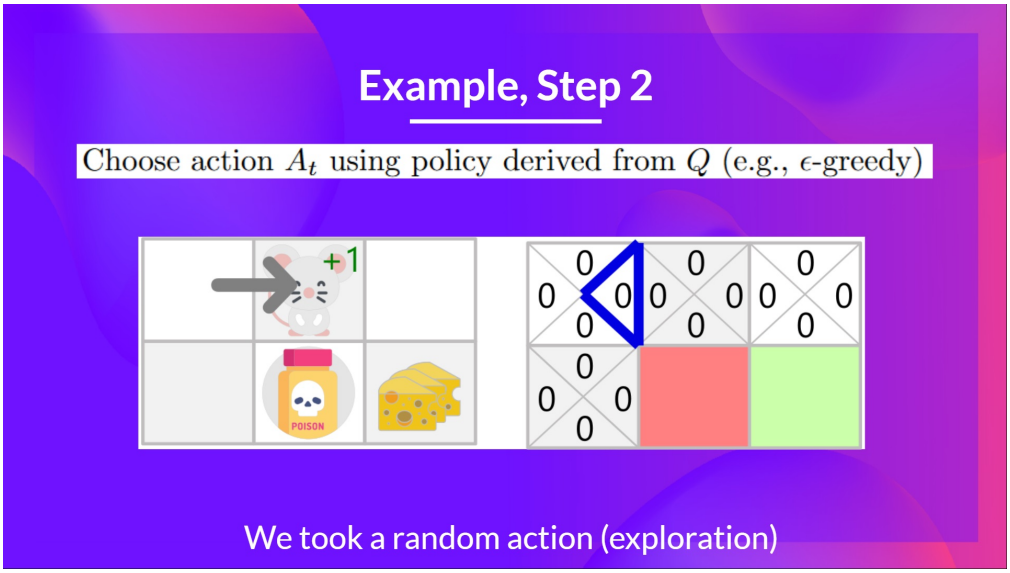


Figure 65: Q-learning example: transition and Q-table update (step 3) [original figure]

9.3 Step 3: Perform action At, get Rt+1 and St+1

By going right, I get a small cheese, so $R_{t+1} = 1$ and I'm in a new state.

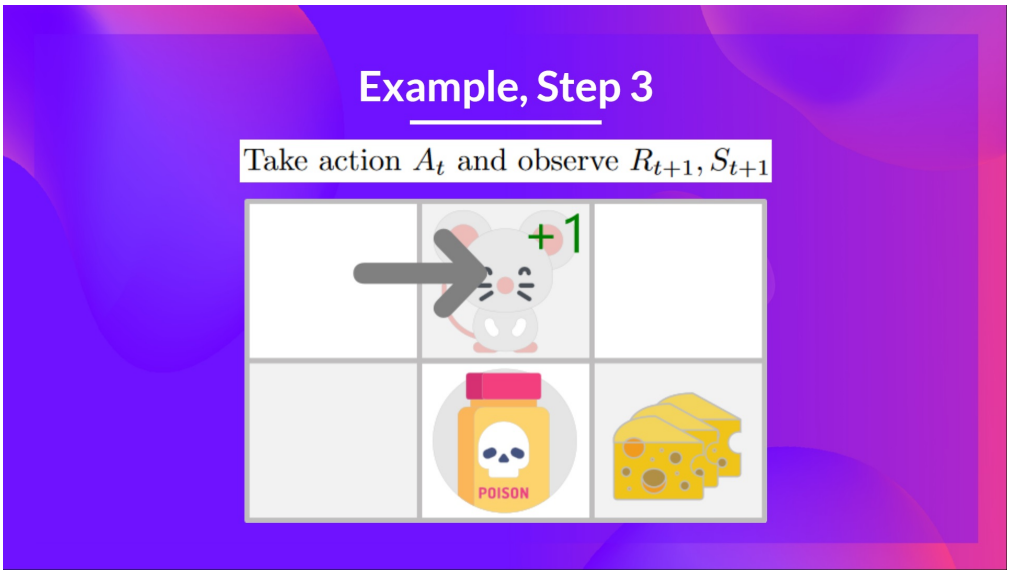


Figure 66: Q-learning example: transition and Q-table update (step 4) [original figure]

9.4 Step 4: Update Q(St, At)

We can now update $Q(S_t, A_t)$ using our formula.

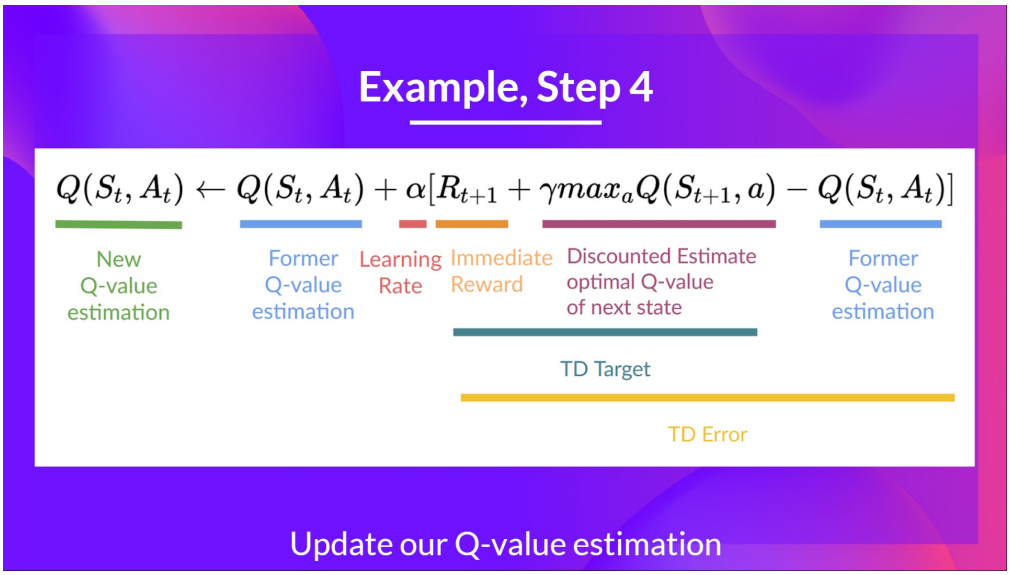


Figure 67: Q-learning example: transition and Q-table update (step 5) [original figure]

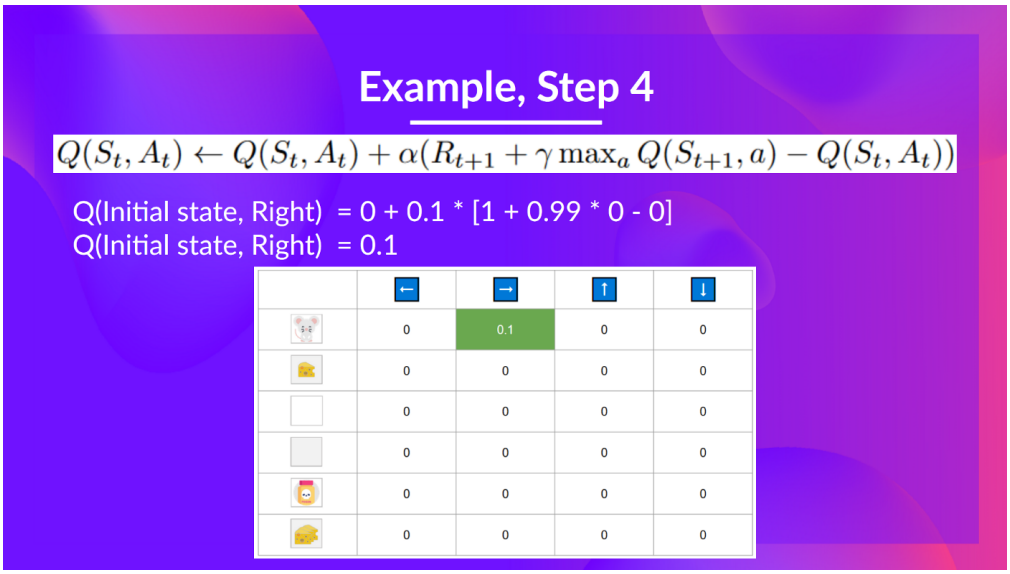


Figure 68: Maze-Example [original figure]

Training timestep 2:

9.5 Step 2: Choose an action using the Epsilon Greedy Strategy

I take a random action again, since epsilon=0.99 is big. (Notice we decay epsilon a little bit because, as the training progress, we want less and less exploration).

I took the action 'down'. **This is not a good action since it leads me to the poison.**

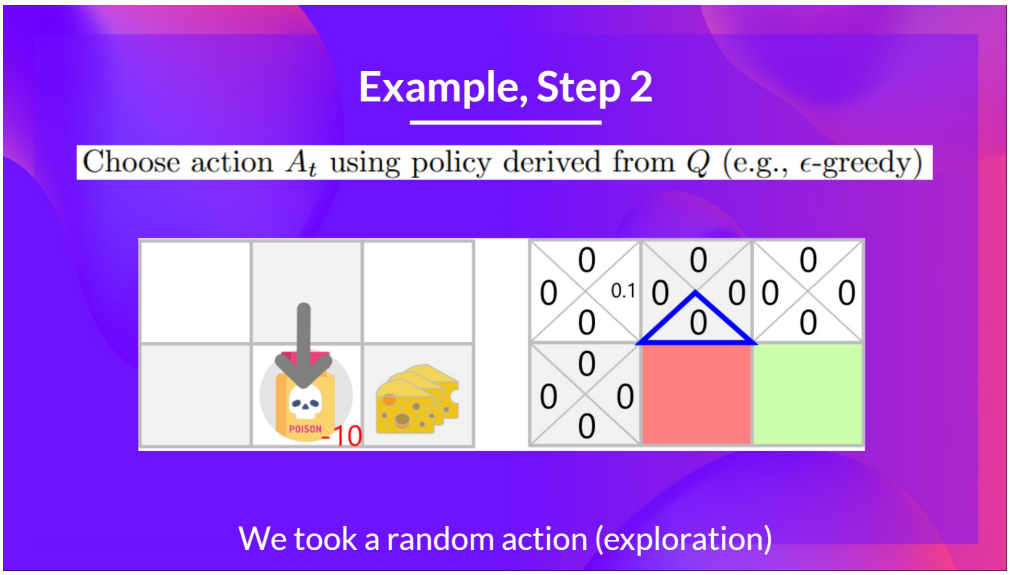


Figure 69: Q-learning example: transition and Q-table update (step 6) [original figure]

9.6 Step 3: Perform action At, get Rt+1 and St+1

Because I ate poison, **I get** $R_{t+1} = -10$**, and I die.**

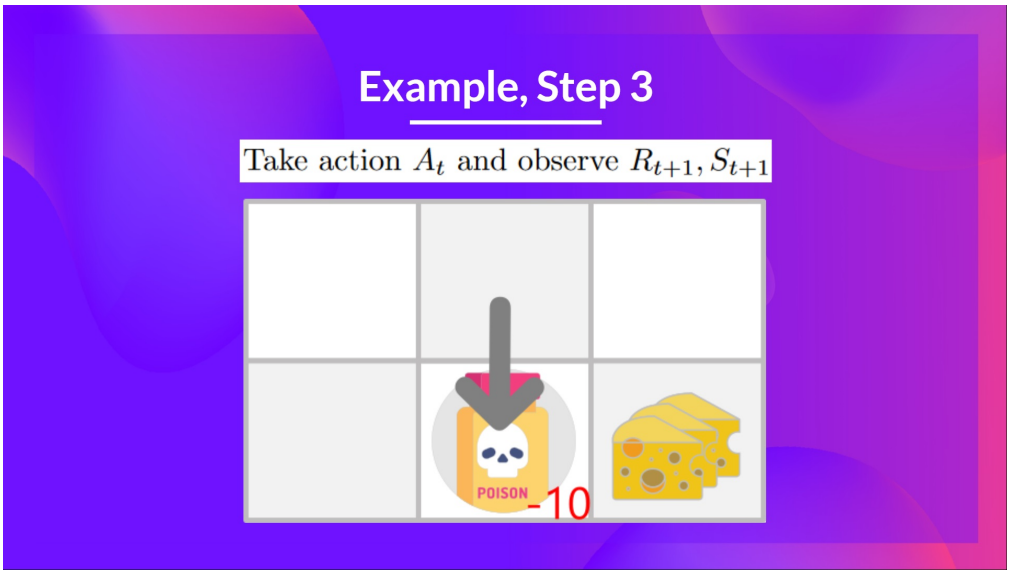


Figure 70: Q-learning example: transition and Q-table update (step 7) [original figure]

9.7 Step 4: Update Q(St, At)

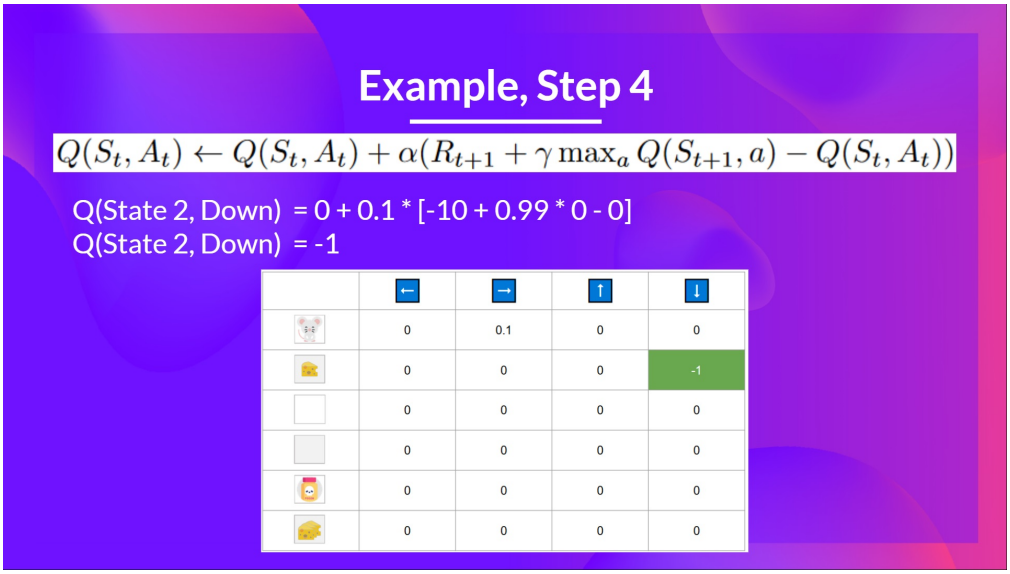


Figure 71: Q-learning example: transition and Q-table update (step 8) [original figure]

Because we're dead, we start a new episode. But what we see here is that, **with two explorations steps, my agent became smarter.**

As we continue exploring and exploiting the environment and updating Q-values using the TD target, the **Q-table will give us a better and better approximation. At the end of the training, we'll get an estimate of the optimal Q-function.**

10 Q-Learning Recap

Q-Learning **is the RL algorithm that** :

- Trains a *Q-function*, an **action-value function** encoded, in internal memory, by a *Q-table* **containing all the state-action pair values.**
- Given a state and action, our Q-function **will search its Q-table for the corresponding value.**

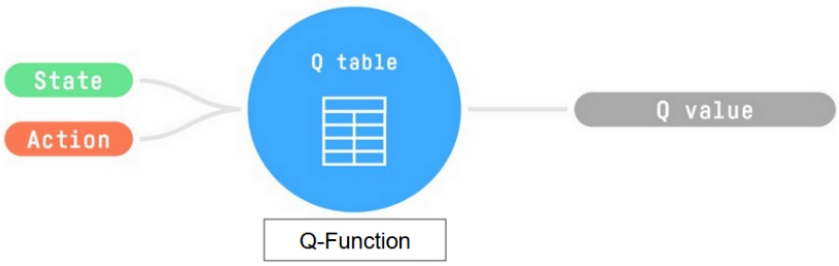


Figure 72: Q-table lookup: obtaining the value of a state-action pair [original figure]

- When the training is done, **we have an optimal Q-function, or, equivalently, an optimal Q-table.**
- And if we **have an optimal Q-function**, we

have an optimal policy, since we **know, for each state, the best action to take.**

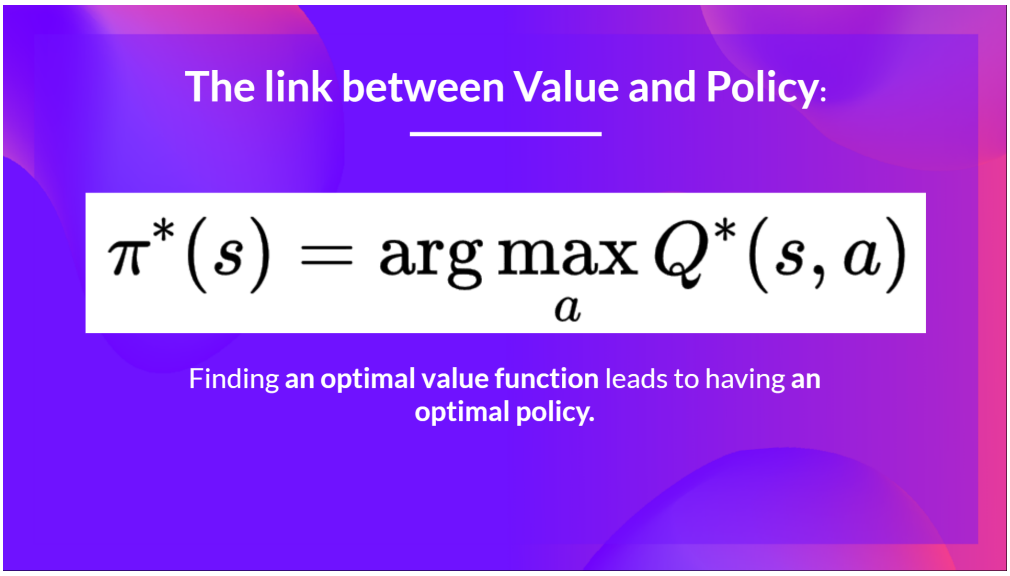


Figure 73: Recovering an optimal policy from the optimal action-value function [original figure]

But, in the beginning, our **Q-table is useless since it gives arbitrary values for each state-action pair (most of the time we initialize the Q-table to 0 values)**. But, as we explore the environment and update our Q-table it will give us a better and better approximation.

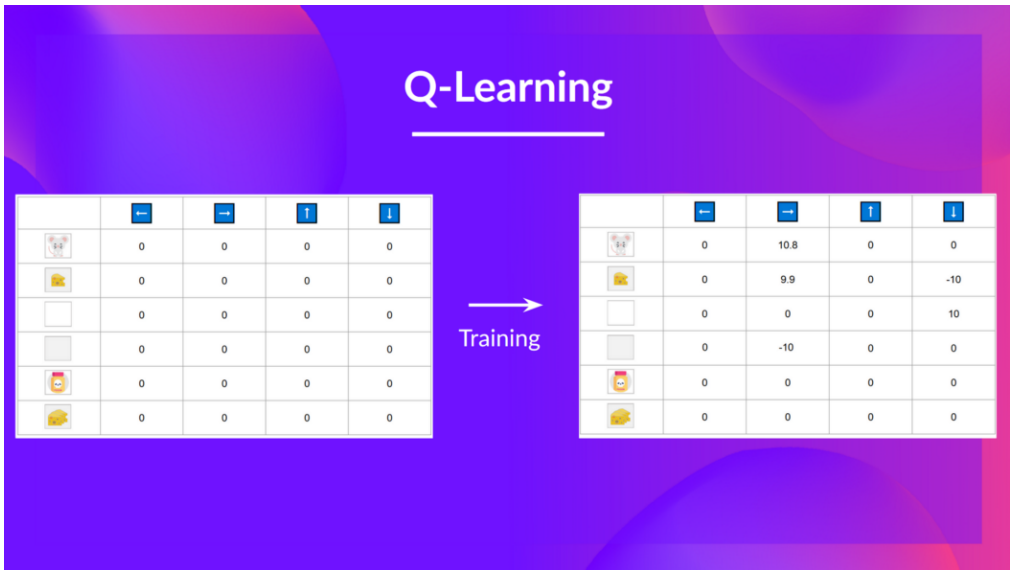


Figure 74: Learning a Q-table from zero initialization toward a useful policy [original figure]

This is the Q-Learning pseudocode:

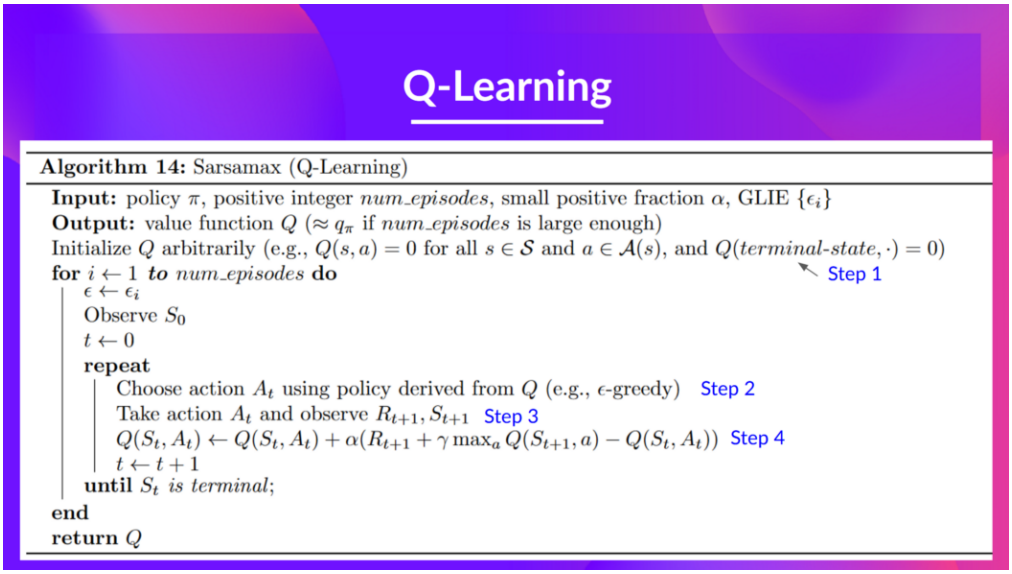


Figure 75: Q-learning: reference algorithm and update pseudocode [original figure]

11 Hands-on

Official practical notebook: https://colab.research.google.com/github/huggingface/deep-rl-class/blob/main/notebooks/unit2/unit2.ipynb

Course help: http://hf.co/join/discord

Now that we studied the Q-Learning algorithm, let's implement it from scratch and train our Q-Learning agent in two environments:

1. Frozen-Lake-v1 (non-slippery and slippery version) : where our agent will need to **go from the starting state (S) to the goal state (G)** by walking only on frozen tiles (F) and avoiding holes (H).
2. An autonomous taxi will need **to learn to navigate** a city to **transport its passengers from point A to point B.**



Figure 76: envs [original animation]

Thanks to a leaderboard, you'll be able to compare your results with other classmates and exchange the best practices to improve your agent's scores. Who will win the challenge for Unit 2?

To validate this hands-on for the certification process, you need to push your trained Taxi model to the Hub and **get a result of >= 4.5**.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

And you can check your progress here https : / / huggingface . co / spaces / ThomasSimonini / Check-my-progress-Deep-RL-Course

To start the hands-on click on the Open In Colab button :

Open original notebook in Google Colab

We strongly **recommend students use Google Colab for the hands-on exercises** instead of running them on their personal computers.

By using Google Colab, **you can focus on learning and experimenting without worrying about the technical aspects** of setting up your environments.

12 Unit 2: Q-Learning with FrozenLake-v1 and Taxi-v3

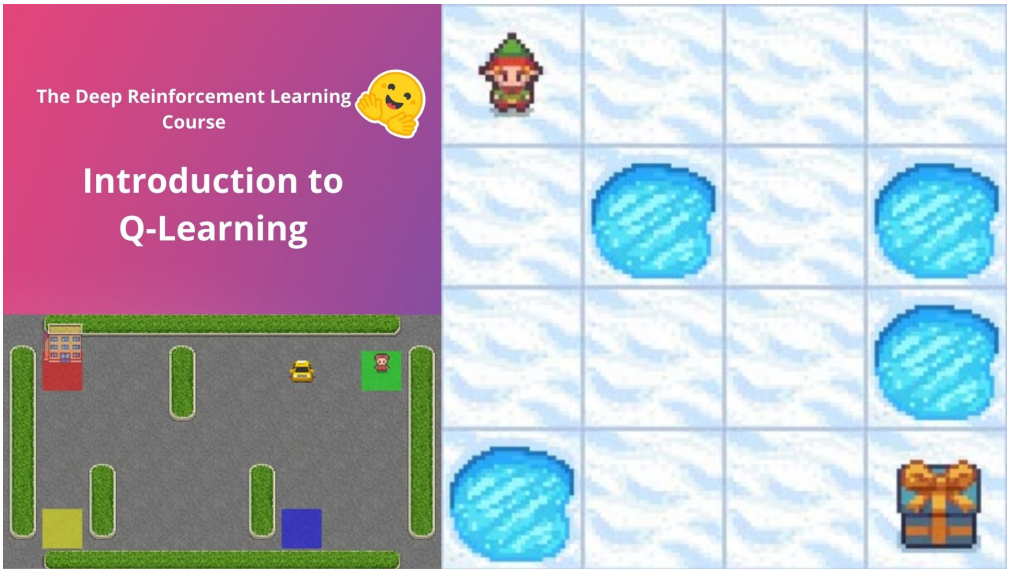


Figure 77: Unit 2 Thumbnail [original figure]

In this notebook, **you'll code your first Reinforcement Learning agent from scratch** to play FrozenLake using Q-Learning, share it with the community, and experiment with different configurations.

Here is an example of what **you will achieve in just a couple of minutes.**

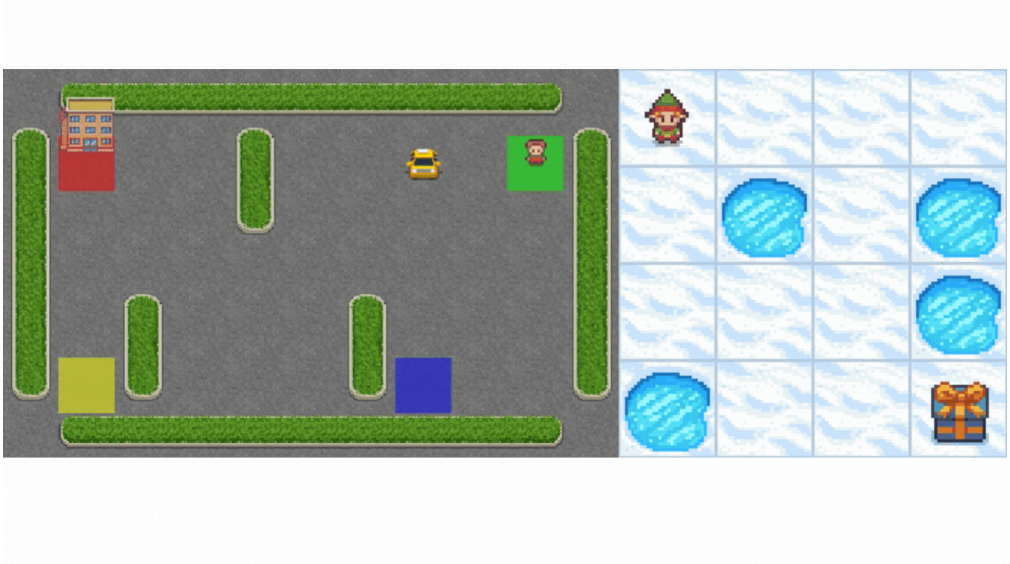


Figure 78: envs [original animation]

Environments:

- FrozenLake-v1
- Taxi-v3

RL-Library:

- Python and NumPy
- Gymnasium

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

12.1 Objectives of this notebook

At the end of the notebook, you will:

- Be able to use **Gymnasium**, the environment library.
- Be able to code a Q-Learning agent from scratch.
- Be able to **push your trained agent and the code to the Hub** with a nice video replay and an evaluation score .

12.2 This notebook is from the Deep Reinforcement Learning Course



Figure 79: Deep RL Course illustration [original figure]

In this free course, you will:

- Study Deep Reinforcement Learning in **theory and practice**.
- Learn to **use famous Deep RL libraries** such as Stable Baselines3, RL Baselines3 Zoo, CleanRL and Sample Factory 2.0.
- Train **agents in unique environments**

And more check the syllabus https://simoninithomas.github.io/deep-rl-course

Don't forget to **sign up to the course** (we are collecting your email to be able to **send you the links when each Unit is published and give you information about the challenges and updates).**

The best way to keep in touch is to join our discord server to exchange with the community and with us https://discord.gg/ydHrjt3WP5

12.3 Prerequisites

Before diving into the notebook, you need to:

Study Q-Learning by reading Unit 2

12.4 A small recap of Q-Learning

Q-Learning **is the RL algorithm that**:

- Trains *Q-Function*, an **action-value function** that is encoded, in internal memory, by a *Q-table* **that contains all the state-action pair values.**
- Given a state and action, our Q-Function **will search the Q-table for the corresponding value.**

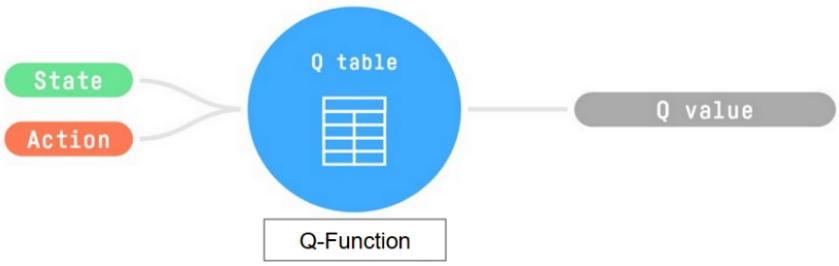


Figure 80: Q-table lookup: obtaining the value of a state-action pair [original figure]

- When the training is done, **we have an optimal Q-Function, so an optimal Q-Table.**
- And if we **have an optimal Q-function**, we

have an optimal policy, since we **know for each state, the best action to take.**

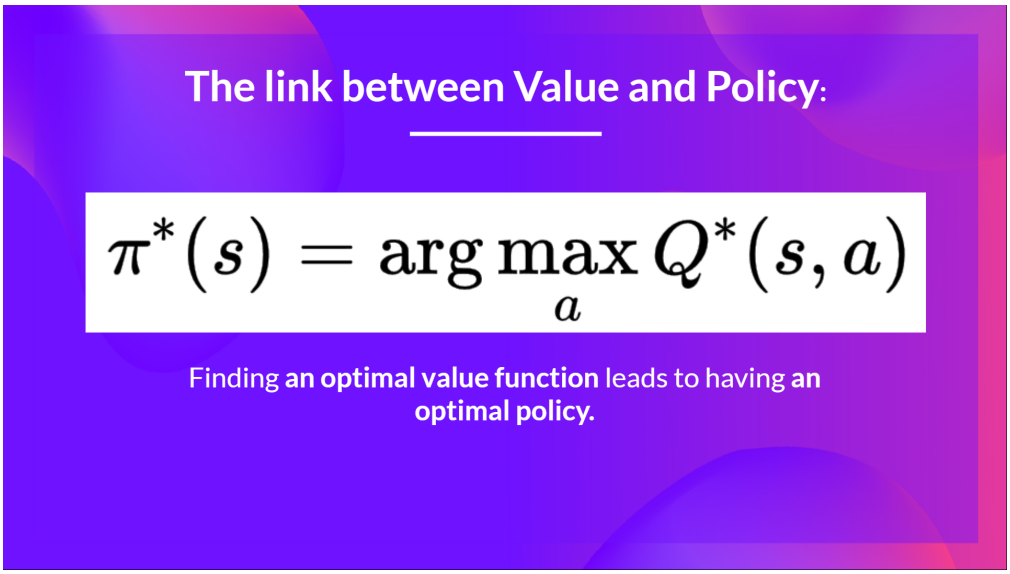


Figure 81: Recovering an optimal policy from the optimal action-value function [original figure]

But, in the beginning, our **Q-Table is useless since it gives arbitrary value for each state-action pair (most of the time we initialize the Q-Table to 0 values)**. But, as we'll explore the environment and update our Q-Table it will give us better and better approximations

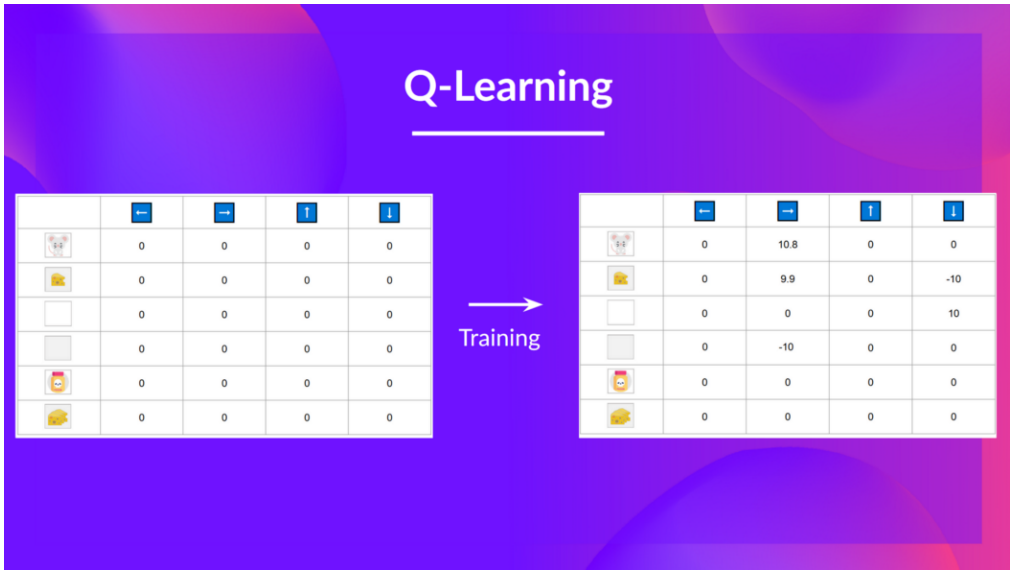


Figure 82: Learning a Q-table from zero initialization toward a useful policy [original figure]

This is the Q-Learning pseudocode:

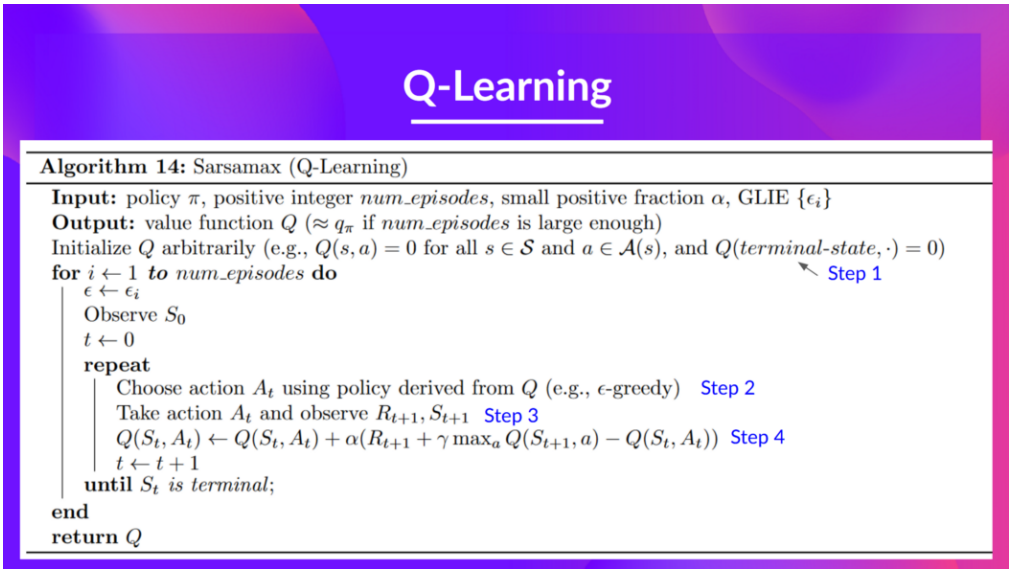


Figure 83: Q-learning: reference algorithm and update pseudocode [original figure]

13 Let's code our first Reinforcement Learning algorithm

To validate this hands-on for the certification process, you need to push your trained Taxi model to the Hub and **get a result of >= 4.5**.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

13.1 Install dependencies and create a virtual display

In the notebook, we'll need to generate a replay video. To do so, with Colab, **we need to have a virtual screen to render the environment** (and thus record the frames).

Hence the following cell will install the libraries and create and run a virtual screen

We'll install multiple ones:

- `gymnasium`: Contains the FrozenLake-v1 and Taxi-v3 environments.
- `pygame`: Used for the FrozenLake-v1 and Taxi-v3 UI.
- `numpy`: Used for handling our Q-table.

The Hugging Face Hub works as a central place where anyone can share and explore models and datasets. It has versioning, metrics, visualizations and other features that will allow you to easily collaborate with others.

You can see here all the Deep RL models available (if they use Q Learning) here https://huggingface.co/models?other=q-learning

```
pip install -r https://raw.githubusercontent.com/huggingface/deep-rl-class/main/notebooks/unit2/
    requirements-unit2.txt
```

```
sudo apt-get update
sudo apt-get install -y python3-opengl
apt install ffmpeg xvfb
pip3 install pyvirtualdisplay
```

To make sure the new installed libraries are used, **sometimes it's required to restart the notebook runtime**. The next cell will force the **runtime to crash, so you'll need to connect again and run the code starting from here**. Thanks to this trick, **we will be able to run our virtual screen.**

```
import os

os.kill(os.getpid(), 9)
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

13.2 Import the packages

In addition to the installed libraries, we also use:

- `random`: To generate random numbers (that will be useful for epsilon-greedy policy).
- `imageio`: To generate a replay video.

```
import numpy as np
import gymnasium as gym
import random
import imageio
import os
import tqdm

import pickle5 as pickle
from tqdm.notebook import tqdm
```

We're now ready to code our Q-Learning algorithm

14 Part 1: Frozen Lake (non slippery version)

14.1 ## Create and understand FrozenLake environment

A good habit when you start to use an environment is to check its documentation

https://gymnasium.farama.org/environments/toy_text/frozen_lake/

We're going to train our Q-Learning agent **to navigate from the starting state (S) to the goal state (G) by walking only on frozen tiles (F) and avoid holes (H)**.

We can have two sizes of environment:

- `map_name="4x4"`: a 4x4 grid version
- `map_name="8x8"`: a 8x8 grid version

The environment has two modes:

- `is_slippery=False`: The agent always moves **in the intended direction** due to the non-slippery nature of the frozen lake (deterministic).

- `is_slippery=True`: The agent **may not always move in the intended direction** due to the slippery nature of the frozen lake (stochastic).

For now let's keep it simple with the 4x4 map and non-slippery. We add a parameter called `render_mode` that specifies how the environment should be visualised. In our case because we **want to record a video of the environment at the end, we need to set render_mode to rgb_array**.

As explained in the documentation "rgb_array": Return a single frame representing the current state of the environment. A frame is a np.ndarray with shape (x, y, 3) representing RGB values for an x-by-y pixel image.

```
# Create the FrozenLake-v1 environment using 4x4 map and non-slippery version and render_mode="
    rgb_array"
env = gym.make()  # TODO use the correct parameters
```

Solution

```
env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=False, render_mode="rgb_array")
```

You can create your own custom grid like this:

```
desc=["SFFF", "FHFH", "FFFH", "HFFG"]
gym.make('FrozenLake-v1', desc=desc, is_slippery=True)
```

but we'll use the default environment for now.

Let's see what the Environment looks like:

```
# We create our environment with gym.make("<name_of_the_environment>")- `is_slippery=False`: The
    agent always moves in the intended direction due to the non-slippery nature of the frozen lake
    (deterministic).
print("_____OBSERVATION SPACE_____ \n")
print("Observation Space", env.observation_space)
print("Sample observation", env.observation_space.sample())  # Get a random observation
```

We see with `Observation Space Shape Discrete(16)` that the observation is an integer representing the **agent's current position as current_row * ncols + current_col (where both the row and col start at 0)**.

For example, the goal position in the 4x4 map can be calculated as follows: 3 * 4 + 3 = 15. The number of possible observations is dependent on the size of the map. **For example, the 4x4 map has 16 possible observations.**

For instance, this is what state = 0 looks like:



Figure 84: FrozenLake grid, actions, and tile types [original figure]

```
print("\n _____ACTION SPACE_____ \n")
print("Action Space Shape", env.action_space.n)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

The action space (the set of possible actions the agent can take) is discrete with 4 actions available :

- 0: GO LEFT
- 1: GO DOWN
- 2: GO RIGHT
- 3: GO UP

Reward function :

- Reach goal: +1
- Reach hole: 0
- Reach frozen: 0

14.2 Create and Initialize the Q-table

(Step 1 of the pseudocode)

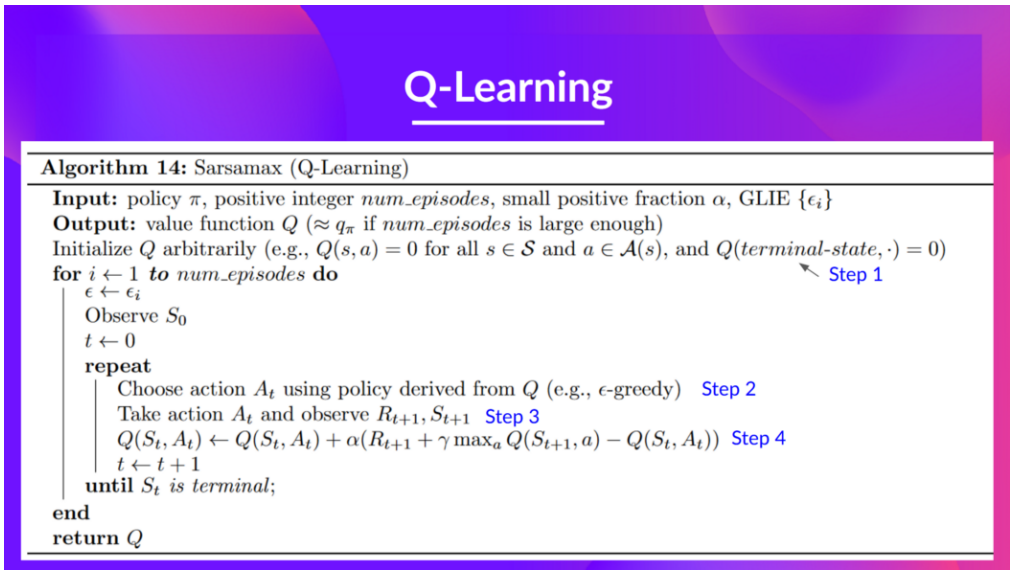


Figure 85: Q-learning: reference algorithm and update pseudocode [original figure]

It's time to initialize our Q-table! To know how many rows (states) and columns (actions) to use, we need to know the action and observation space. We already know their values from before, but we'll want to obtain them programmatically so that our algorithm generalizes for different environments. Gym provides us a way to do that: `env.action_space.n` and `env.observation_space.n`

```
state_space =
print("There are ", state_space, " possible states")

action_space =
print("There are ", action_space, " possible actions")
```

```
# Let's create our Qtable of size (state_space, action_space) and initialized each values at 0
    using np.zeros. np.zeros needs a tuple (a,b)
def initialize_q_table(state_space, action_space):
  Qtable =
  return Qtable
```

```
Qtable_frozenlake = initialize_q_table(state_space, action_space)
```

Solution

```
state_space = env.observation_space.n
print("There are ", state_space, " possible states")

action_space = env.action_space.n
print("There are ", action_space, " possible actions")
```

```
# Let's create our Qtable of size (state_space, action_space) and initialized each values at 0
    using np.zeros
def initialize_q_table(state_space, action_space):
    Qtable = np.zeros((state_space, action_space))
    return Qtable
```

```
Qtable_frozenlake = initialize_q_table(state_space, action_space)
```

14.3 Define the greedy policy

Remember we have two policies since Q-Learning is an **off-policy** algorithm. This means we're using a **different policy for acting and updating the value function**.

- Epsilon-greedy policy (acting policy)
- Greedy-policy (updating policy)

The greedy policy will also be the final policy we'll have when the Q-learning agent completes training. The greedy policy is used to select an action using the Q-table.

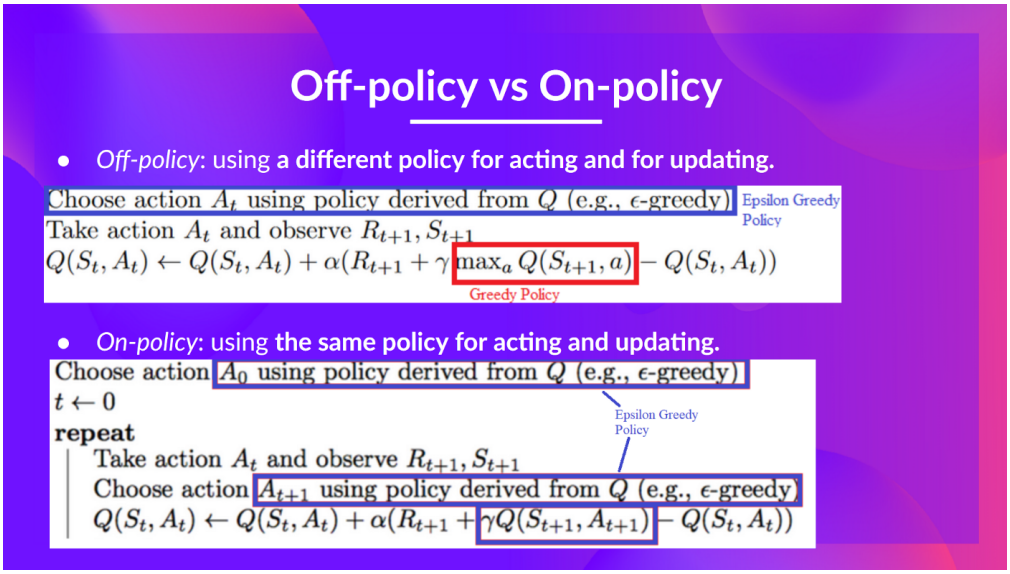


Figure 86: Comparison of off-policy and on-policy methods [original figure]

```
def greedy_policy(Qtable, state):
  # Exploitation: take the action with the highest state, action value
  action =

  return action
```

Solution

```
def greedy_policy(Qtable, state):
    # Exploitation: take the action with the highest state, action value
    action = np.argmax(Qtable[state][:])

    return action
```

14.4 Define the epsilon-greedy policy

Epsilon-greedy is the training policy that handles the exploration/exploitation trade-off.

The idea with epsilon-greedy:

- With *probability 1 - ε* : **we do exploitation** (i.e. our agent selects the action with the highest state-action pair value).
- With *probability ε*: we do **exploration** (trying a random action).

As the training continues, we progressively **reduce the epsilon value since we will need less and less exploration and more exploitation.**

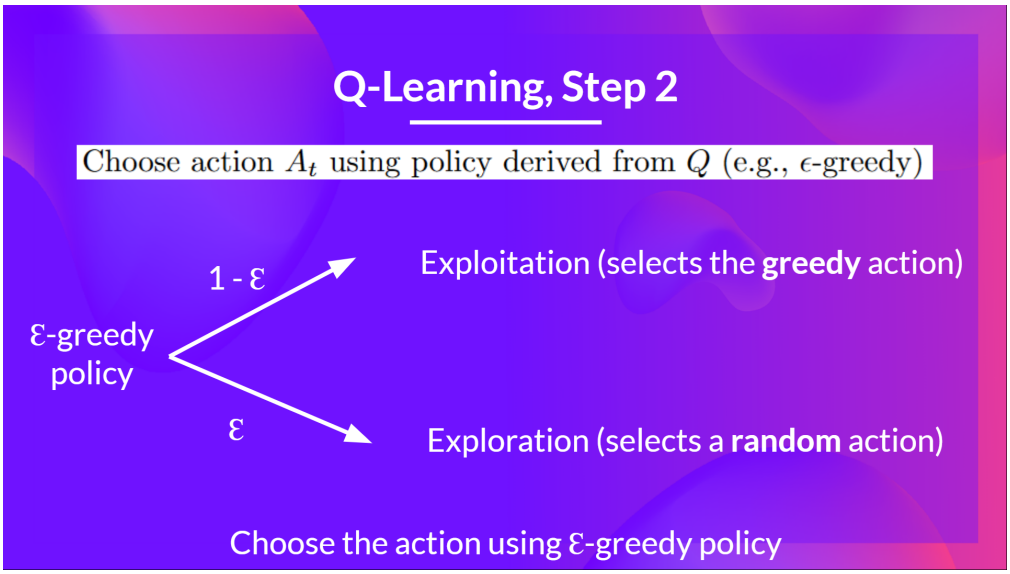


Figure 87: Epsilon-greedy strategy: exploration and exploitation [original figure]

```
def epsilon_greedy_policy(Qtable, state, epsilon):
  # Randomly generate a number between 0 and 1
  random_num =
  # if random_num > greater than epsilon --> exploitation
  if random_num > epsilon:
    # Take the action with the highest value given a state
    # np.argmax can be useful here
    action =
  # else --> exploration
  else:
    action = # Take a random action

  return action
```

Solution

```
def epsilon_greedy_policy(Qtable, state, epsilon):
    # Randomly generate a number between 0 and 1
    random_num = random.uniform(0, 1)
    # if random_num > greater than epsilon --> exploitation
    if random_num > epsilon:
        # Take the action with the highest value given a state
        # np.argmax can be useful here
        action = greedy_policy(Qtable, state)
    # else --> exploration
    else:
        action = env.action_space.sample()

    return action
```

14.5 Define the hyperparameters

The exploration related hyperparamters are some of the most important ones.

- We need to make sure that our agent **explores enough of the state space** to learn a good value approximation. To do that, we need to have progressive decay of the epsilon.
- If you decrease epsilon too fast (too high decay_rate), **you take the risk that your agent will be stuck**, since your agent didn't explore enough of the state space and hence can't solve the problem.

```
# Training parameters
n_training_episodes = 10000  # Total training episodes
learning_rate = 0.7  # Learning rate

# Evaluation parameters
n_eval_episodes = 100  # Total number of test episodes

# Environment parameters
env_id = "FrozenLake-v1"  # Name of the environment
max_steps = 99  # Max steps per episode
gamma = 0.95  # Discounting rate
eval_seed = []  # The evaluation seed of the environment

# Exploration parameters
max_epsilon = 1.0  # Exploration probability at start
min_epsilon = 0.05  # Minimum exploration probability
decay_rate = 0.0005  # Exponential decay rate for exploration prob
```

14.6 Create the training loop method

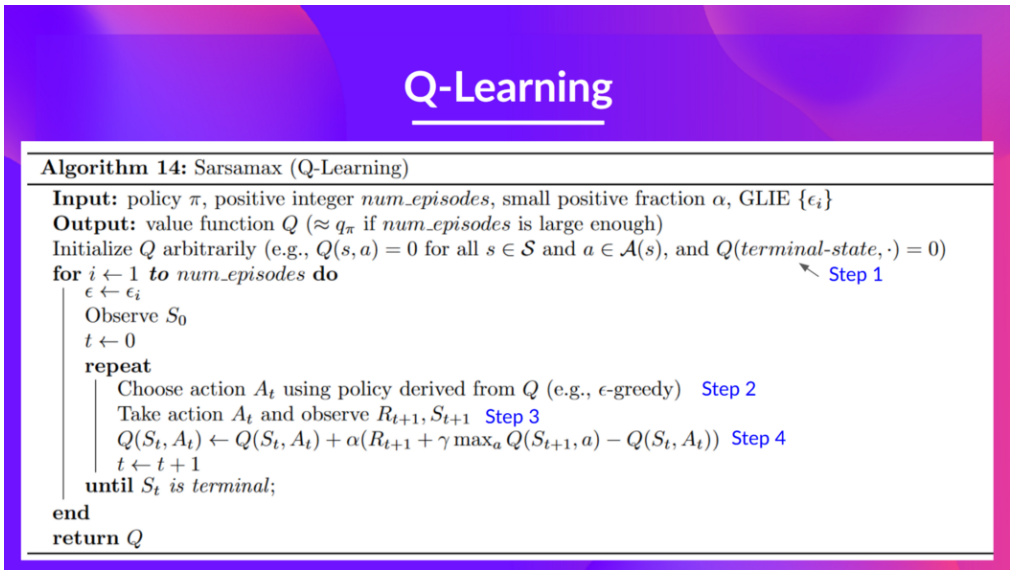


Figure 88: Q-learning: reference algorithm and update pseudocode [original figure]

The training loop goes like this:

```
For episode in the total of training episodes:

Reduce epsilon (since we need less and less exploration)
Reset the environment

  For step in max timesteps:
    Choose the action At using epsilon greedy policy
    Take the action (a) and observe the outcome state(s') and reward (r)
    Update the Q-value Q(s,a) using Bellman equation Q(s,a) + lr [R(s,a) + gamma * max Q(s',a') - Q
    (s,a)]
    If done, finish the episode
    Our next state is the new state
```

```
def train(n_training_episodes, min_epsilon, max_epsilon, decay_rate, env, max_steps, Qtable):
  for episode in tqdm(range(n_training_episodes)):
    # Reduce epsilon (because we need less and less exploration)
    epsilon = min_epsilon + (max_epsilon - min_epsilon)*np.exp(-decay_rate*episode)
```

```
    # Reset the environment
    state, info = env.reset()
    step = 0
    terminated = False
    truncated = False

    # repeat
    for step in range(max_steps):
      # Choose the action At using epsilon greedy policy
      action =

      # Take action At and observe Rt+1 and St+1
      # Take the action (a) and observe the outcome state(s') and reward (r)
      new_state, reward, terminated, truncated, info =

      # Update Q(s,a):= Q(s,a) + lr [R(s,a) + gamma * max Q(s',a') - Q(s,a)]
      Qtable[state][action] =

      # If terminated or truncated finish the episode
      if terminated or truncated:
        break

      # Our next state is the new state
      state = new_state
  return Qtable
```

Solution

```
def train(n_training_episodes, min_epsilon, max_epsilon, decay_rate, env, max_steps, Qtable):
    for episode in tqdm(range(n_training_episodes)):
        # Reduce epsilon (because we need less and less exploration)
        epsilon = min_epsilon + (max_epsilon - min_epsilon) * np.exp(-decay_rate * episode)
        # Reset the environment
        state, info = env.reset()
        step = 0
        terminated = False
        truncated = False

        # repeat
        for step in range(max_steps):
            # Choose the action At using epsilon greedy policy
            action = epsilon_greedy_policy(Qtable, state, epsilon)

            # Take action At and observe Rt+1 and St+1
            # Take the action (a) and observe the outcome state(s') and reward (r)
            new_state, reward, terminated, truncated, info = env.step(action)

            # Update Q(s,a):= Q(s,a) + lr [R(s,a) + gamma * max Q(s',a') - Q(s,a)]
            Qtable[state][action] = Qtable[state][action] + learning_rate * (
                reward + gamma * np.max(Qtable[new_state]) - Qtable[state][action]
            )

            # If terminated or truncated finish the episode
            if terminated or truncated:
                break

            # Our next state is the new state
            state = new_state
    return Qtable
```

14.7 Train the Q-Learning agent

```
Qtable_frozenlake = train(n_training_episodes, min_epsilon, max_epsilon, decay_rate, env, max_steps,
     Qtable_frozenlake)
```

14.8 Let's see what our Q-Learning table looks like now

```
Qtable_frozenlake
```

14.9 The evaluation method

- We defined the evaluation method that we're going to use to test our Q-Learning agent.

```
def evaluate_agent(env, max_steps, n_eval_episodes, Q, seed):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and std of
    reward.
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param Q: The Q-table
    :param seed: The evaluation seed array (for taxi-v3)
    """
    episode_rewards = []
    for episode in tqdm(range(n_eval_episodes)):
        if seed:
            state, info = env.reset(seed=seed[episode])
        else:
            state, info = env.reset()
        step = 0
        truncated = False
        terminated = False
        total_rewards_ep = 0

        for step in range(max_steps):
            # Take the action (index) that have the maximum expected future reward given that state
            action = greedy_policy(Q, state)
            new_state, reward, terminated, truncated, info = env.step(action)
            total_rewards_ep += reward

            if terminated or truncated:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward
```

14.10 Evaluate our Q-Learning agent

- Usually, you should have a mean reward of 1.0
- The **environment is relatively easy** since the state space is really small (16). What you can try to do is to replace it with the slippery version, which introduces stochasticity, making the environment more complex.

```
# Evaluate our Agent
mean_reward, std_reward = evaluate_agent(env, max_steps, n_eval_episodes, Qtable_frozenlake,
    eval_seed)
print(f"Mean_reward={mean_reward:.2f} +/- {std_reward:.2f}")
```

14.11 Publish our trained model to the Hub

Now that we saw good results after the training, **we can publish our trained model to the Hub with one line of code**.

Here's an example of a Model Card:

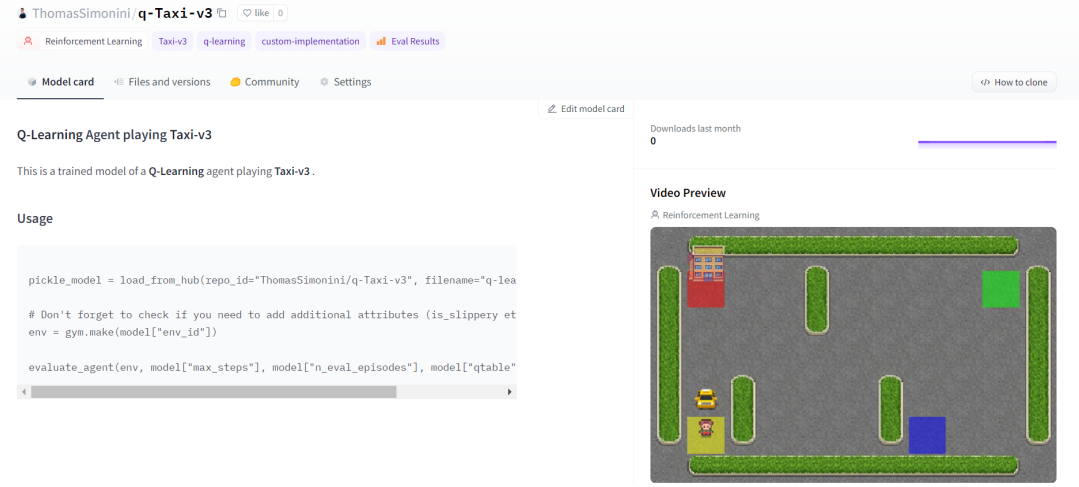


Figure 89: Model card [original figure]

Under the hood, the Hub uses git-based repositories (don't worry if you don't know what git is), which means you can update the model with new versions as you experiment and improve your agent.

Do not modify this code

```
from huggingface_hub import HfApi, snapshot_download
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import json
```

```
def record_video(env, Qtable, out_directory, fps=1):
    """
    Generate a replay video of the agent
    :param env
    :param Qtable: Qtable of our agent
    :param out_directory
    :param fps: how many frame per seconds (with taxi-v3 and frozenlake-v1 we use 1)
    """
    images = []
    terminated = False
    truncated = False
    state, info = env.reset(seed=random.randint(0, 500))
    img = env.render()
    images.append(img)
    while not terminated or truncated:
        # Take the action (index) that have the maximum expected future reward given that state
        action = np.argmax(Qtable[state][:])
        state, reward, terminated, truncated, info = env.step(
            action
        )  # We directly put next_state = state for recording logic
        img = env.render()
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=fps)
```

```
def push_to_hub(repo_id, model, env, video_fps=1, local_repo_path="hub"):
    """
    Evaluate, Generate a video and Upload a model to Hugging Face Hub.
    This method does the complete pipeline:
    - It evaluates the model
    - It generates the model card
    - It generates a replay video of the agent
    - It pushes everything to the Hub

    :param repo_id: repo_id: id of the model repository from the Hugging Face Hub
    :param env
    :param video_fps: how many frame per seconds to record our video replay
    (with taxi-v3 and frozenlake-v1 we use 1)
    :param local_repo_path: where the local repository is
    """
    _, repo_name = repo_id.split("/")

    eval_env = env
    api = HfApi()

    # Step 1: Create the repo
    repo_url = api.create_repo(
        repo_id=repo_id,
        exist_ok=True,
    )

    # Step 2: Download files
    repo_local_path = Path(snapshot_download(repo_id=repo_id))

    # Step 3: Save the model
    if env.spec.kwargs.get("map_name"):
        model["map_name"] = env.spec.kwargs.get("map_name")
        if env.spec.kwargs.get("is_slippery", "") == False:
            model["slippery"] = False

    # Pickle the model
    with open((repo_local_path) / "q-learning.pkl", "wb") as f:
        pickle.dump(model, f)

    # Step 4: Evaluate the model and build JSON with evaluation metrics
    mean_reward, std_reward = evaluate_agent(
        eval_env, model["max_steps"], model["n_eval_episodes"], model["qtable"], model["eval_seed"]
    )

    evaluate_data = {
        "env_id": model["env_id"],
        "mean_reward": mean_reward,
        "n_eval_episodes": model["n_eval_episodes"],
        "eval_datetime": datetime.datetime.now().isoformat(),
    }

    # Write a JSON file called "results.json" that will contain the
    # evaluation results
    with open(repo_local_path / "results.json", "w") as outfile:
        json.dump(evaluate_data, outfile)

    # Step 5: Create the model card
    env_name = model["env_id"]
    if env.spec.kwargs.get("map_name"):
        env_name += "-" + env.spec.kwargs.get("map_name")

    if env.spec.kwargs.get("is_slippery", "") == False:
        env_name += "-" + "no_slippery"
```

```
  metadata = {}
  metadata["tags"] = [env_name, "q-learning", "reinforcement-learning", "custom-implementation"]

  # Add metrics
  eval = metadata_eval_result(
      model_pretty_name=repo_name,
      task_pretty_name="reinforcement-learning",
      task_id="reinforcement-learning",
      metrics_pretty_name="mean_reward",
      metrics_id="mean_reward",
      metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
      dataset_pretty_name=env_name,
      dataset_id=env_name,
  )

  # Merges both dictionaries
  metadata = {**metadata, **eval}

  model_card = f"""
# **Q-Learning** Agent playing1 **{env_id}**
This is a trained model of a **Q-Learning** agent playing **{env_id}** .

## Usage

model = load_from_hub(repo_id="{repo_id}", filename="q-learning.pkl")

# Don't forget to check if you need to add additional attributes (is_slippery=False etc)
env = gym.make(model["env_id"])
"""

  evaluate_agent(env, model["max_steps"], model["n_eval_episodes"], model["qtable"], model["
  eval_seed"])

  readme_path = repo_local_path / "README.md"
  readme = ""
  print(readme_path.exists())
  if readme_path.exists():
      with readme_path.open("r", encoding="utf8") as f:
          readme = f.read()
  else:
      readme = model_card

  with readme_path.open("w", encoding="utf-8") as f:
      f.write(readme)

  # Save our metrics to Readme metadata
  metadata_save(readme_path, metadata)

  # Step 6: Record a video
  video_path = repo_local_path / "replay.mp4"
  record_video(env, model["qtable"], video_path, video_fps)

  # Step 7. Push everything to the Hub
  api.upload_folder(
      repo_id=repo_id,
      folder_path=repo_local_path,
      path_in_repo=".",
  )

  print("Your model is pushed to the Hub. You can view your model here: ", repo_url)
```

.

By using `push_to_hub` **you evaluate, record a replay, generate a model card of your agent and push it to the Hub**.

This way:

- You can **showcase our work**
- You can **visualize your agent playing**
- You can **share an agent with the community that others can use**
- You can **access a leaderboard to see how well your agent is performing compared to your classmates** https : / / huggingface . co / spaces / huggingface-projects / Deep-Reinforcement-Learning-Leaderboard

To be able to share your model with the community there are three more steps to follow:

1 (If it's not already done) create an account to HF https://huggingface.co/join

2 Sign in and then, you need to store your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**

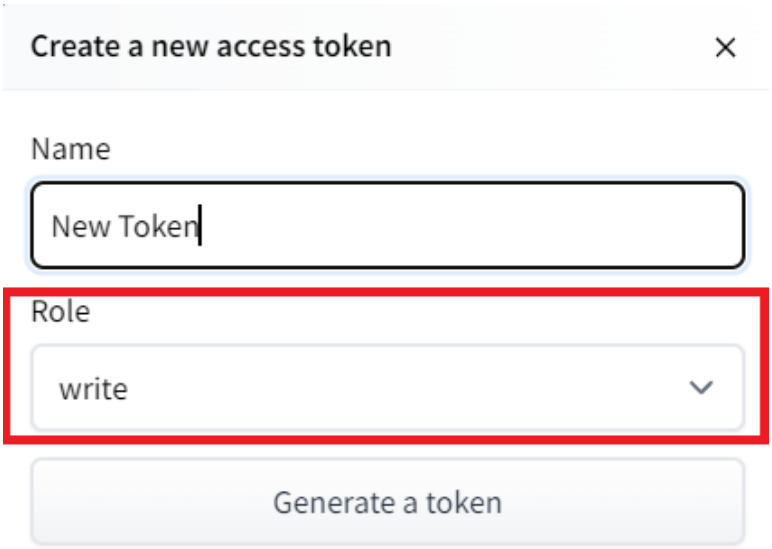


Figure 90: Generating a Hugging Face Hub access token [original figure]

```
from huggingface_hub import notebook_login

notebook_login()
```

If you don't want to use a Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login` (or `login`)

3 We're now ready to push our trained agent to the Hub using `push_to_hub()` function

- Let's create **the model dictionary that contains the hyperparameters and the Q_table**.

```
model = {
    "env_id": env_id,
    "max_steps": max_steps,
    "n_training_episodes": n_training_episodes,
    "n_eval_episodes": n_eval_episodes,
    "eval_seed": eval_seed,
    "learning_rate": learning_rate,
    "gamma": gamma,
```

```
    "max_epsilon": max_epsilon,
    "min_epsilon": min_epsilon,
    "decay_rate": decay_rate,
    "qtable": Qtable_frozenlake,
}
```

Let's fill the `push_to_hub` function:

- `repo_id`: the name of the Hugging Face Hub Repository that will be created/updated ‘

(repo_id = {username}/{repo_name})`A good`repo_id`is`{username}/q-{env_id}‘

- `model`: our model dictionary containing the hyperparameters and the Qtable.
- `env`: the environment.
- `commit_message`: message of the commit

```
model
```

```
username = ""  # FILL THIS
repo_name = "q-FrozenLake-v1-4x4-noSlippery"
push_to_hub(repo_id=f"{username}/{repo_name}", model=model, env=env)
```

Congrats you've just implemented from scratch, trained, and uploaded your first Reinforcement Learning agent. FrozenLake-v1 no_slippery is very simple environment, let's try a harder one .

15 Part 2: Taxi-v3

15.1 ## Create and understand Taxi-v3

A good habit when you start to use an environment is to check its documentation

https://gymnasium.farama.org/environments/toy_text/taxi/

In `Taxi-v3` , there are four designated locations in the grid world indicated by R(ed), G(reen), Y(ellow), and B(lue).

When the episode starts, **the taxi starts off at a random square** and the passenger is at a random location. The taxi drives to the passenger's location, **picks up the passenger**, drives to the passenger's destination (another one of the four specified locations), and then **drops off the passenger**. Once the passenger is dropped off, the episode ends.

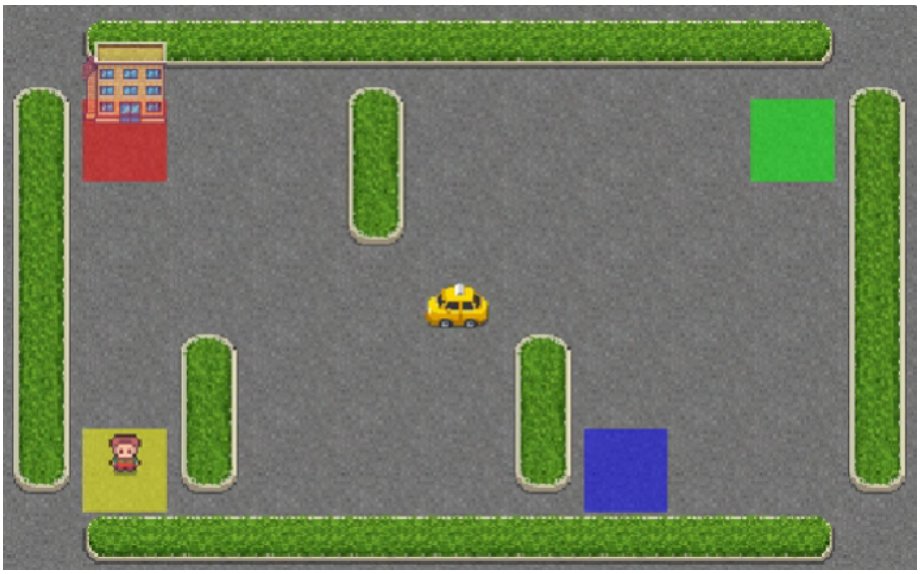


Figure 91: Taxi environment used for Q-learning [original figure]

```
env = gym.make("Taxi-v3", render_mode="rgb_array")
```

There are **500 discrete states since there are 25 taxi positions, 5 possible locations of the passenger** (including the case when the passenger is in the taxi), and **4 destination locations.**

```
state_space = env.observation_space.n
print("There are ", state_space, " possible states")
```

```
action_space = env.action_space.n
print("There are ", action_space, " possible actions")
```

The action space (the set of possible actions the agent can take) is discrete with **6 actions available** :

- 0: move south
- 1: move north
- 2: move east
- 3: move west
- 4: pickup passenger
- 5: drop off passenger

Reward function :

- -1 per step unless other reward is triggered.
- +20 delivering passenger.
- -10 executing “pickup” and “drop-off” actions illegally.

```
# Create our Q table with state_size rows and action_size columns (500x6)
Qtable_taxi = initialize_q_table(state_space, action_space)
print(Qtable_taxi)
print("Q-table shape: ", Qtable_taxi.shape)
```

15.2 Define the hyperparameters

DO NOT MODIFY EVAL_SEED: the eval_seed array **allows us to evaluate your agent with the same taxi starting positions for every classmate**

```
# Training parameters
n_training_episodes = 25000  # Total training episodes
learning_rate = 0.7  # Learning rate

# Evaluation parameters
n_eval_episodes = 100  # Total number of test episodes

# DO NOT MODIFY EVAL_SEED
eval_seed = [
    16,
    54,
    165,
    177,
    191,
    191,
    120,
    80,
    149,
    178,
    48,
    38,
    6,
    125,
    174,
    73,
    50,
    172,
    100,
    148,
    146,
    6,
    25,
    40,
    68,
    148,
    49,
    167,
    9,
    97,
    164,
    176,
    61,
    7,
    54,
    55,
    161,
    131,
    184,
    51,
    170,
    12,
    120,
    113,
    95,
    126,
    51,
    98,
    36,
    135,
    54,
```

```
    82,
    45,
    95,
    89,
    59,
    95,
    124,
    9,
    113,
    58,
    85,
    51,
    134,
    121,
    169,
    105,
    21,
    30,
    11,
    50,
    65,
    12,
    43,
    82,
    145,
    152,
    97,
    106,
    55,
    31,
    85,
    38,
    112,
    102,
    168,
    123,
    97,
    21,
    83,
    158,
    26,
    80,
    63,
    5,
    81,
    32,
    11,
    28,
    148,
]  # Evaluation seed, this ensures that all classmates agents are trained on the same taxi starting
     position
# Each seed has a specific starting state
# Environment parameters
env_id = "Taxi-v3"  # Name of the environment
max_steps = 99  # Max steps per episode
gamma = 0.95  # Discounting rate

# Exploration parameters
max_epsilon = 1.0  # Exploration probability at start
min_epsilon = 0.05  # Minimum exploration probability
decay_rate = 0.005  # Exponential decay rate for exploration prob
```

15.3 Train our Q-Learning agent

```
Qtable_taxi = train(n_training_episodes, min_epsilon, max_epsilon, decay_rate, env, max_steps,
    Qtable_taxi)
Qtable_taxi
```

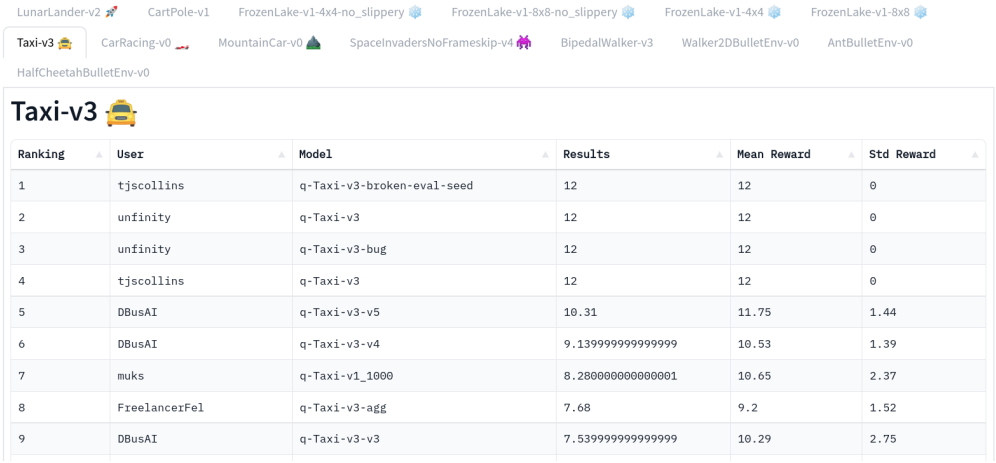
15.4 Create a model dictionary and publish our trained model to the Hub

- We create a model dictionary that will contain all the training hyperparameters for reproducibility and the Q-Table.

```
model = {
    "env_id": env_id,
    "max_steps": max_steps,
    "n_training_episodes": n_training_episodes,
    "n_eval_episodes": n_eval_episodes,
    "eval_seed": eval_seed,
    "learning_rate": learning_rate,
    "gamma": gamma,
    "max_epsilon": max_epsilon,
    "min_epsilon": min_epsilon,
    "decay_rate": decay_rate,
    "qtable": Qtable_taxi,
}
```

```
username = ""  # FILL THIS
repo_name = ""  # FILL THIS
push_to_hub(repo_id=f"{username}/{repo_name}", model=model, env=env)
```

Now that it's on the Hub, you can compare the results of your Taxi-v3 with your classmates using the leaderboard https : / / huggingface . co / spaces / huggingface-projects / Deep-Reinforcement-Learning-Leaderboard



Ranking	User	Model	Results	Mean Reward	Std Reward
1	tjscollins	q-Taxi-v3-broken-eval-seed	12	12	0
2	unfinity	q-Taxi-v3	12	12	0
3	unfinity	q-Taxi-v3-bug	12	12	0
4	tjscollins	q-Taxi-v3	12	12	0
5	DBusAI	q-Taxi-v3-v5	10.31	11.75	1.44
6	DBusAI	q-Taxi-v3-v4	9.139999999999999	10.53	1.39
7	muks	q-Taxi-v1_1000	8.280000000000001	10.65	2.37
8	FreelancerFel	q-Taxi-v3-agg	7.68	9.2	1.52
9	DBusAI	q-Taxi-v3-v3	7.539999999999999	10.29	2.75

Figure 92: Taxi Leaderboard [original figure]

16 Part 3: Load from Hub

What's amazing with Hugging Face Hub is that you can easily load powerful models from the community.

Loading a saved model from the Hub is really easy:

1. You go https://huggingface.co/models?other=q-learning to see the list of all the q-learning

saved models.
2. You select one and copy its repo_id

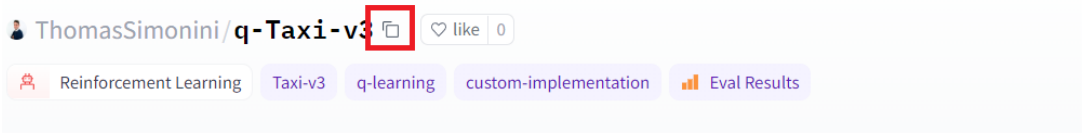


Figure 93: Example of locating a model repository ID on Hugging Face Hub [original figure]

3. Then we just need to use `load_from_hub` with:

- The repo_id
- The filename: the saved model inside the repo.

Do not modify this code

```
from urllib.error import HTTPError

from huggingface_hub import hf_hub_download


def load_from_hub(repo_id: str, filename: str) -> str:
    """
    Download a model from Hugging Face Hub.
    :param repo_id: id of the model repository from the Hugging Face Hub
    :param filename: name of the model zip file from the repository
    """
    # Get the model from the Hub, download and cache the model on your local disk
    pickle_model = hf_hub_download(repo_id=repo_id, filename=filename)

    with open(pickle_model, "rb") as f:
        downloaded_model_file = pickle.load(f)

    return downloaded_model_file
```

.

```
model = load_from_hub(repo_id="ThomasSimonini/q-Taxi-v3", filename="q-learning.pkl")  # Try to use
    another model

print(model)
env = gym.make(model["env_id"])

evaluate_agent(env, model["max_steps"], model["n_eval_episodes"], model["qtable"], model["eval_seed
    "])
```

```
model = load_from_hub(
    repo_id="ThomasSimonini/q-FrozenLake-v1-no-slippery", filename="q-learning.pkl"
)  # Try to use another model

env = gym.make(model["env_id"], is_slippery=False)

evaluate_agent(env, model["max_steps"], model["n_eval_episodes"], model["qtable"], model["eval_seed
    "])
```

16.1 Some additional challenges

The best way to learn **is to try things on your own**! As you saw, the current agent is not doing great. As a first suggestion, you can train for more steps. With 1,000,000 steps, we saw some great results!

In the Leaderboard you will find your agents. Can you get to the top?

Here are some ideas to climb up the leaderboard:

- Train more steps
- Try different hyperparameters by looking at what your classmates have done.
- **Push your new trained model** on the Hub

Are walking on ice and driving taxis too boring to you? Try to **change the environment**, why not use FrozenLake-v1 slippery version? Check how they work using the gymnasium documentation and have fun .

Congrats , you've just implemented, trained, and uploaded your first Reinforcement Learning agent.

Understanding Q-Learning is an **important step to understanding value-based methods.**

In the next Unit with Deep Q-Learning, we'll see that while creating and updating a Q-table was a good strategy — **however, it is not scalable.**

For instance, imagine you create an agent that learns to play Doom.

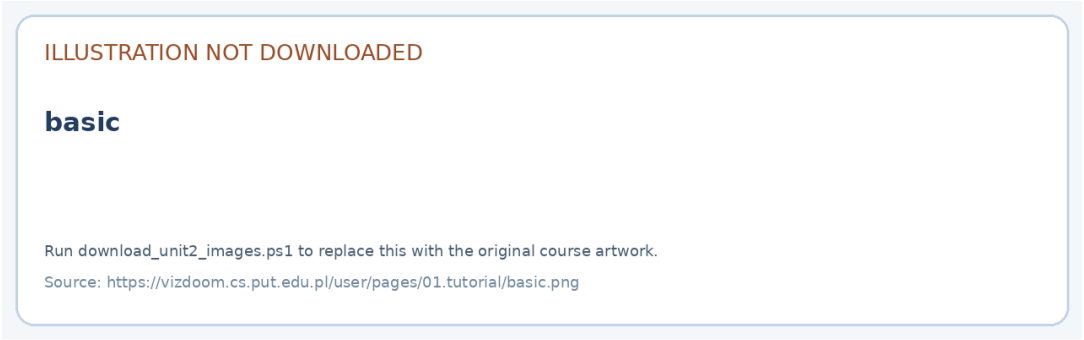


Figure 94: Doom [original figure]

Doom is a large environment with a huge state space (millions of different states). Creating and updating a Q-table for that environment would not be efficient.

That's why we'll study Deep Q-Learning in the next unit, an algorithm **where we use a neural network that approximates, given a state, the different Q-values for each action.**

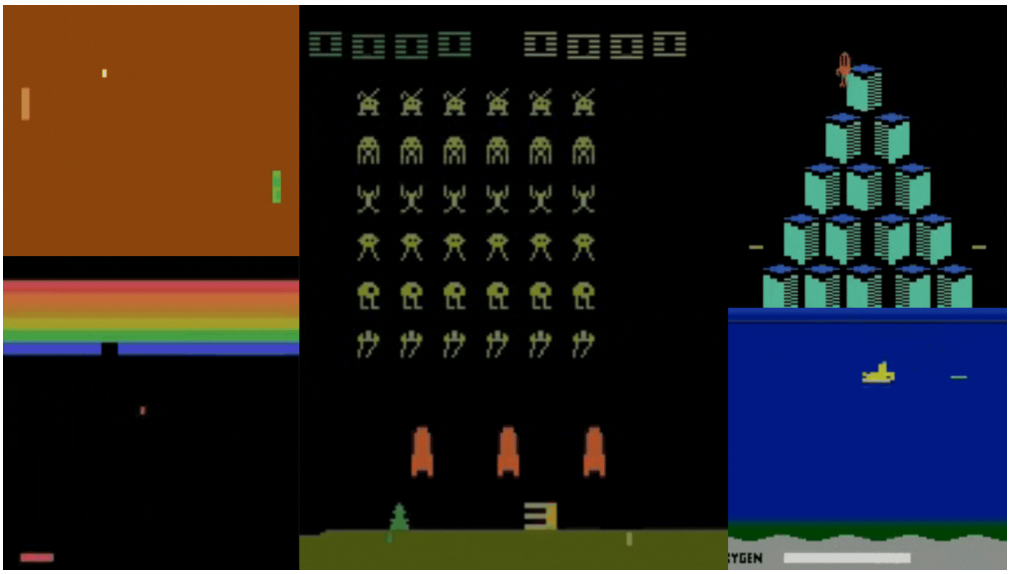


Figure 95: atari envs [original animation]

See you in Unit 3!

16.2 Keep learning, stay awesome

17 Second Quiz

The best way to learn and to avoid the illusion of competence **is to test yourself.** This will help you to find **where you need to reinforce your knowledge**.

Q1: What is Q-Learning?

Select all applicable answers.

- **A.** The algorithm we use to train our Q-function
- **B.** A value function
- **C.** An algorithm that determines the value of being at a particular state and taking a specific action at that state
- **D.** A table

Q2: What is a Q-table?

Select all applicable answers.

- **A.** An algorithm we use in Q-Learning
- **B.** Q-table is the internal memory of our agent
- **C.** In Q-table each cell corresponds a state value

Q3: Why if we have an optimal Q-function Q^* we have an optimal policy?

Q4: Can you explain what is Epsilon-Greedy Strategy?

Q5: How do we update the Q value of a state, action pair?

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Figure 96: Quiz: identify terms in the Q-learning update rule [original figure]

Q6: What's the difference between on-policy and off-policy

17.1 Answer Key and Explanations

Q1: What is Q-Learning?

Correct choice(s): A

- **Option B:** It's an action-value function since it determines the value of being at a particular state and taking a specific action at that state
- **Option C:** Q-function is the function that determines the value of being at a particular state and taking a specific action at that state.
- **Option D:** Q-learning is not a Q-table. The Q-function is the algorithm that will feed the Q-table.

Q2: What is a Q-table?

Correct choice(s): B

- **Option C:** Each cell corresponds to a state-action value pair value. Not a state value.

Q3: Why if we have an optimal Q-function Q^* we have an optimal policy?

Because if we have an optimal Q-function, we have an optimal policy since we know for each state what is the best action to take.

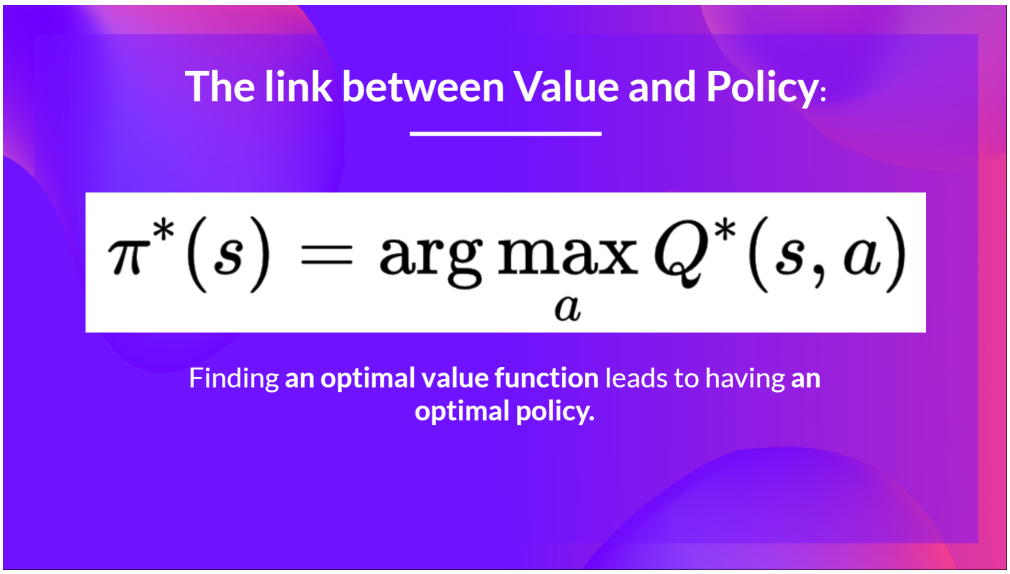


Figure 97: Recovering an optimal policy from the optimal action-value function [original figure]

Q4: Can you explain what is Epsilon-Greedy Strategy?

Epsilon Greedy Strategy is a policy that handles the exploration/exploitation trade-off.

The idea is that we define epsilon $\varepsilon = 1.0$:

- With *probability 1 — ε :* we do exploitation (aka our agent selects the action with the highest state-action pair value).
- With *probability ε :* we do exploration (trying random action).

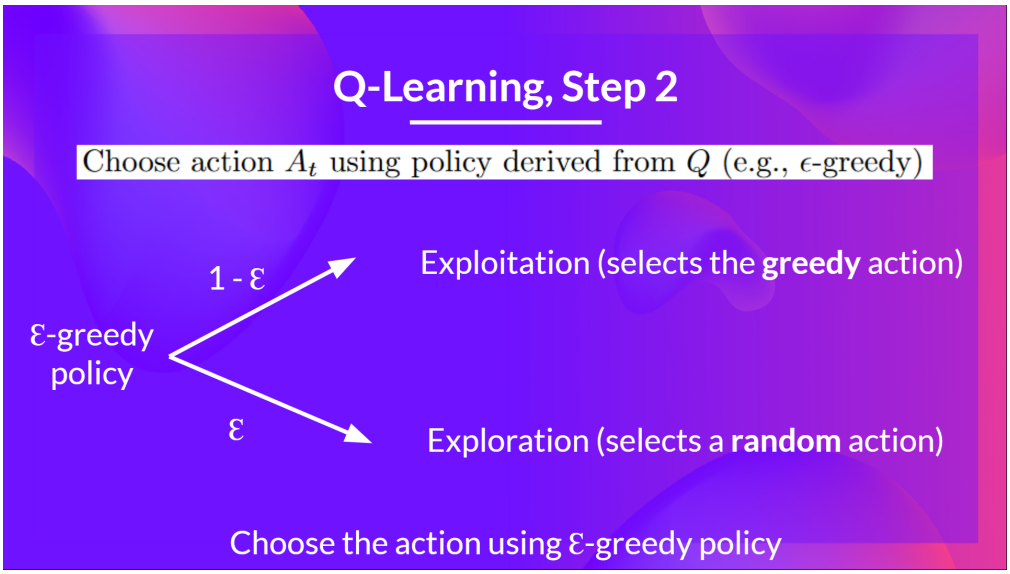


Figure 98: Epsilon-greedy strategy: exploration and exploitation [original figure]

Q5: How do we update the Q value of a state, action pair?

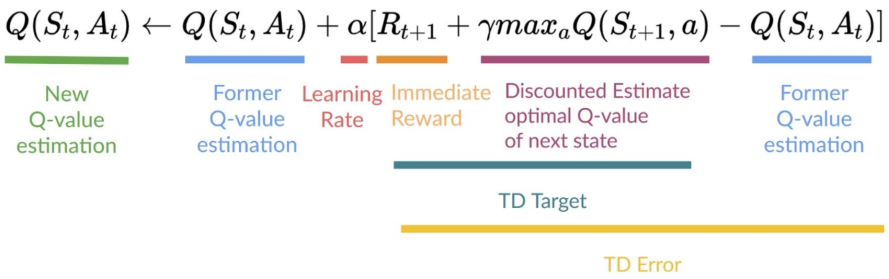


Figure 99: Solution: terms in the Q-learning update rule [original figure]

Q6: What's the difference between on-policy and off-policy

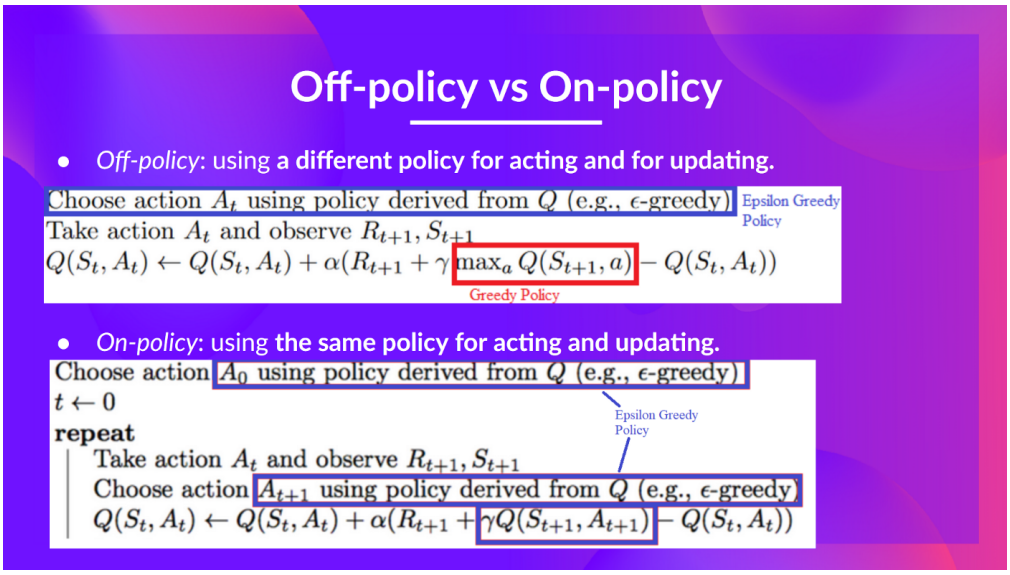


Figure 100: Comparison of off-policy and on-policy methods [original figure]

Congrats on finishing this Quiz , if you missed some elements, take time to read again the chapter to reinforce () your knowledge.

18 Glossary

This is a community-created glossary. Contributions are welcomed!

Strategies to find the optimal policy

- **Policy-based methods.** The policy is usually trained with a neural network to select what action to take given a state. In this case it is the neural network which outputs the action that the agent should take instead of using a value function. Depending on the experience received by the environment, the neural network will be re-adjusted and will provide better actions.
- **Value-based methods.** In this case, a value function is trained to output the value of a state or a state-action pair that will represent our policy. However, this value doesn't define what

action the agent should take. In contrast, we need to specify the behavior of the agent given the output of the value function. For example, we could decide to adopt a policy to take the action that always leads to the biggest reward (Greedy Policy). In summary, the policy is a Greedy Policy (or whatever decision the user takes) that uses the values of the value-function to decide the actions to take.

Among the value-based methods, we can find two main strategies

- **The state-value function.** For each state, the state-value function is the expected return if the agent starts in that state and follows the policy until the end.
- **The action-value function.** In contrast to the state-value function, the action-value calculates for each state and action pair the expected return if the agent starts in that state, takes that action, and then follows the policy forever after.

Epsilon-greedy strategy:

- Common strategy used in reinforcement learning that involves balancing exploration and exploitation.
- Chooses the action with the highest expected reward with a probability of 1-epsilon.
- Chooses a random action with a probability of epsilon.
- Epsilon is typically decreased over time to shift focus towards exploitation.

Greedy strategy:

- Involves always choosing the action that is expected to lead to the highest reward, based on the current knowledge of the environment. (Only exploitation)
- Always chooses the action with the highest expected reward.
- Does not include any exploration.
- Can be disadvantageous in environments with uncertainty or unknown optimal actions.

Off-policy vs on-policy algorithms

- **Off-policy algorithms:** A different policy is used at training time and inference time
- **On-policy algorithms:** The same policy is used during training and inference

Monte Carlo and Temporal Difference learning strategies

- **Monte Carlo (MC):** Learning at the end of the episode. With Monte Carlo, we wait until the episode ends and then we update the value function (or policy function) from a complete episode.
- **Temporal Difference (TD):** Learning at each step. With Temporal Difference Learning, we update the value function (or policy function) at each step without requiring a complete episode.

If you want to improve the course, you can open a Pull Request.

This glossary was made possible thanks to:

- Ramón Rueda
- Hasarindu Perera
- Arkady Arkhangorodsky

19 Conclusion

Congrats on finishing this chapter! There was a lot of information. And congrats on finishing the tutorials. You've just implemented your first RL agent from scratch and shared it on the Hub .

Implementing from scratch when you study a new architecture **is important to understand how it works.**

It's **normal if you still feel confused** by all these elements. **This was the same for me and for everyone who studies RL.**

Take time to really grasp the material before continuing.

In the next chapter, we're going to dive deeper by studying our first Deep Reinforcement Learning algorithm based on Q-Learning: Deep Q-Learning. And you'll train a **DQN agent with RL-Baselines3 Zoo to play Atari Games**.

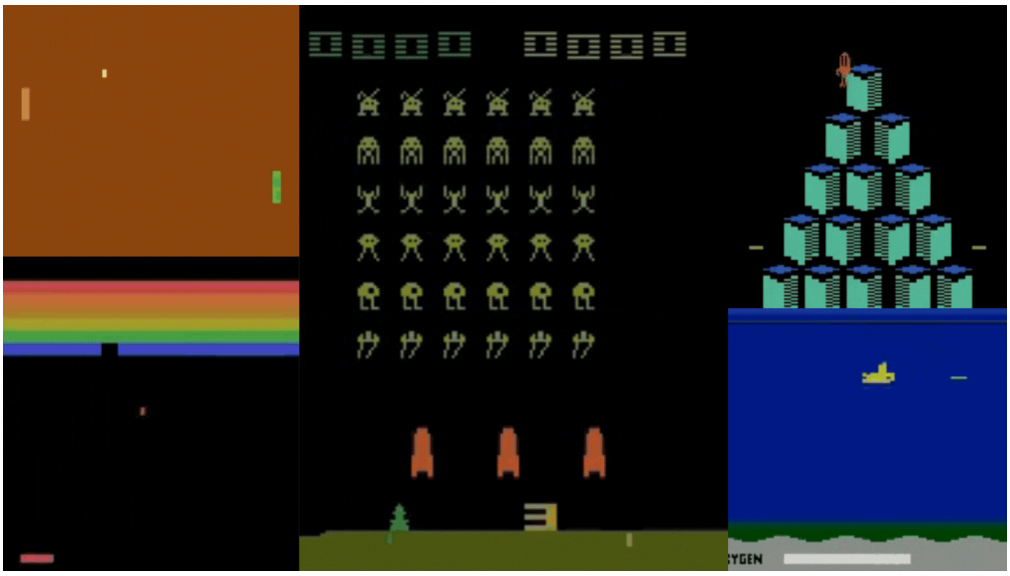


Figure 101: Atari environments [original animation]

Finally, we would love **to hear what you think of the course and how we can improve it**. If you have some feedback then please fill this form

Keep Learning, stay awesome

20 Additional Readings

These are **optional readings** if you want to go deeper.

20.1 Monte Carlo and TD Learning

To dive deeper into Monte Carlo and Temporal Difference Learning:

- Why do temporal difference (TD) methods have lower variance than Monte Carlo methods?
- When are Monte Carlo methods preferred over temporal difference ones?

20.2 Q-Learning

- Reinforcement Learning: An Introduction, Richard Sutton and Andrew G. Barto Chapter 5, 6 and 7
- Foundations of Deep RL Series, L2 Deep Q-Learning by Pieter Abbeel